Department of Electrical and Computer Engineering
University of Toronto

ECE496Y Design Project Course
Group Final Report

Title: VIVIDpdf (Very Intelligent Vocal Interpreter for Digesting PDF)		
Team #:	2025188	
Team members:	Nathan Hung	nathan.hung@mail.utoronto.ca
	Henry Li	hl.li@mail.utoronto.ca
	Xiaoxing Liu	xiaoxing.liu@mail.utoronto.ca
	Joshua Chau	joshuatt.chau@mail.utoronto.ca
Supervisor(s):	Professor Belinda Wang	
Section #:	1	
Administrator:	Professor Hamid Timorabadi	
Submission Date:	21st of March, 2026	

Group Final Report Attribution Table

This table should be filled out to accurately reflect who contributed to each section of the report and what they contributed. Provide a **column** for each student, a **row** for each major section of the report, and the appropriate codes (e.g., 'RD,' 'MR') in each of the necessary cells in the table. You may expand the table, inserting rows as needed, but you should not require more than two pages. The original completed and signed form must be included in the hard copies of the final report. Please make a copy of it for your own reference.

Section #	**Student Names**			
	Nathan Hung	Henry Li	Xiaoxing Liu	Joshua Chau
Exective Summary	MR	ET	RD	ET
Table of Content	ET	MR	ET	RD
1.0	ET	MR	ET	RD
1.1	RS, ET	MR	ET	RS, RD
1.2	RS, ET	MR	ET	RS, RD
1.3	RS, ET	MR	ET	RS, RD
2.0	ET	RS, RD	RS, ET	MR
2.1	ET	RS, RD	RS, ET	MR
2.2	ET	RS, RD	RS, ET	MR
2.3	ET	RS, RD	RS, ET	MR
2.4	ET	RS, RD	RS, ET	MR
3.0	RS, RD, MR	ET	RS, RD	ET
3.1	RS, RD	ET	MR	ET
4.0	MR	ET	ET	RD
5.0	ET	MR	ET	RD
6.0	RS, RD, ET, MR	RS, RD, ET	RS, RD, ET	RS, RD, ET
All	FP	FP	CM	CM

Abbreviation Codes:

Fill in abbreviations for roles for each of the required content elements. You do not have to fill in every cell. The "**All**" row refers to the complete report and should indicate who was responsible for the final compilation and final read-through of the completed document.

RS – responsible for research of information
RD – wrote the first draft
MR – responsible for major revision
ET – edited for grammar, spelling, and expression
OR – other
"All" row abbreviations:
FP – final read-through of complete document for flow and consistency
CM – responsible for compiling the elements into the complete document
OR - other

Signatures

By signing below, you verify that you have read the attribution table and agree that it accurately reflects your contribution to this document.

Name:	Nathan Hung	**Signature:**	Nathan Hung	**Date:**	2026-03-21
Name:	Henry Li	**Signature:**	Henry Li	**Date:**	2026-03-21
Name:	Xiaoxing Liu	**Signature:**	Xiaoxing Liu	**Date:**	2026-03-21
Name:	Joshua Chau	**Signature:**	Tsz Tung Chau	**Date:**	2026-03-21

Voluntary Document Release Consent Form

To all ECE496 students:

To better help future students, we would like to provide examples that are drawn from excerpts of past student reports. The examples will be used to illustrate general communication principles as well as how the document guidelines can be applied to a variety of categories of design projects (e.g., electronics, computers, software, networking, and research).

Any material chosen for the examples will be altered so that all names are removed. In addition, where possible, many of the technical details will also be removed so that the structure or presentation style is highlighted rather than the original technical content. These examples will be made available to students on the course website and, in general, may be accessible by the public. The original reports will not be released but will be accessible only to the course instructors and administrative staff.

Participation is completely voluntary, and students may refuse to participate or may withdraw their permission at any time. Reports will only be used with the signed consent of all team members. Participating will have no influence on the grading of your work, and there is no penalty for not taking part.

If your group agrees to take part, please have all members sign the bottom of this form.

Consent Statement
We verify that we have read the above letter and are giving permission for the ECE496 course coordinator to use our reports as outlined above.

Team #: 2025188 Project Title: Intelligent Text-To-Speech

Supervisor(s): Professor Belinda Wang Administrator: Professor Hamid Timorabadi

Name:	Nathan Hung	**Signature:**	Nathan Hung	**Date:**	2026-03-17
Name:	Henry Li	**Signature:**	Henry Li	**Date:**	2026-03-21
Name:	Xiaoxing Liu	**Signature:**	Xiaoxing Liu	**Date:**	2026-03-21
Name:	Joshua Chau	**Signature:**	Tsz Tung Chau	**Date:**	2026-03-17

Group Highlights (Author: Nathan)

VIVIDpdf is a web-based text-to-speech tool made specifically for academic PDFs. Standard reading apps often struggle with technical papers because they misread math and read aloud unwanted text like page numbers or citations. VIVIDpdf solves this by giving users control over what is read and how it sounds. It runs completely in the web browser, meaning users do not need to download an app or upload their private files to a server.

The team built a PDF reader app with features that fix common problems for students reading dense documents:

- **Intelligent math reading**: The app uses AI to look at math equations and read them naturally (for example, reading "x^3" as "x cubed").
- **Element Skipping**: Users can draw boxes on the page to skip headers, footers, and charts. The app also automatically skips URLs, emails, and citations so the audio does not get interrupted.
- **Custom Words**: Users can set custom rules to fix bad pronunciations, like telling the app to read “pmos” as “P moss” rather than “P-M-O-S”.
- **Click-to-Read**: Users can click any word on the page to start the audio from that exact spot. The text highlights on the screen as it reads aloud.
- **Large File Support**: The app can easily open and read very large PDF files (< 1GB).

The final product successfully met all 11 of its mandatory requirements, 2 objectives, and 3 constraints.

- **High Performance**: The app scored a 99 for speed and a perfect 100 for accessibility on Google PageSpeed Insights.
- **Works Anywhere**: It runs smoothly on Windows, Mac, and iPads. It also works on all major web browsers, including Chrome, Safari, and Firefox.
- **Real-World Success**: During a public beta test, the site had over 200 visits and users read over 500 sentences. It also helped one team member (Nathan) listen to more than 200 pages of coursework during the semester.
- **On Budget**: The team kept costs low. The final cost was $158.91 CAD, staying well under their $250 budget.

Individual Contributions

Nathan

Nathan was primarily responsible for designing and implementing the core PDF reader and document-interaction framework of VIVIDpdf, establishing the system's visual and functional foundation. Given the need for precise synchronization between document content, highlighting, and audio playback, the reader was developed as a stable platform for higher-level features rather than a simple file viewer.

He implemented the full PDF loading and rendering pipeline using PDF.js, including drag-and-drop and file selection uploads, and multi-page rendering. He also developed key navigation and viewing functionalities, including page jumping, manual zoom control, fit-to-width and fit-to-height modes, and page rotation. These features ensured compatibility with a diverse range of academic and technical documents.

Nathan further improved usability and performance through features such as automatic scrolling during playback, dark mode support, and mobile pinch-to-zoom. To optimize rendering efficiency for large documents, he introduced lazy loading via the Intersection Observer API and handled device-pixel-ratio scaling for high-resolution displays.

In addition, he contributed to the underlying viewer architecture required for features such as highlighting, click-to-read, and AI-assisted reading. This involved resolving layout inconsistencies, ensuring alignment across zoom levels and rotations, and maintaining synchronization of overlay-based interactions. Overall, his work delivered a stable, responsive, and extensible reading environment and supported system integration, debugging, and final validation.

Henry

Henry was primarily responsible for designing and implementing the text-to-speech pipeline and text parsing logic in VIVIDpdf. His work focused on transforming raw PDF and OCR-extracted text into structured, playable content that could be naturally read aloud and reliably synchronized with the document display.

He integrated the Web Speech API and developed core playback functionalities, including play, pause, stop, and adjustable playback speed. He also implemented multi-language and multi-voice support, prioritizing voice selection to ensure consistency across browsers and operating systems, and automatic fallback to English voices when necessary to maintain usability.

A key technical contribution was the development of robust sentence segmentation logic. To address inconsistencies in extracted text, he designed parsing rules that extended beyond standard punctuation, incorporating font-size changes, line breaks, paragraph spacing, colon-based structures, and capitalization patterns. Additional handling for hyphen merging, vertical alignment tolerance, and cross-page sentence continuation significantly improved the continuity and readability of spoken output, particularly for complex academic and OCR-heavy documents.

Henry also supported both sentence-level and word-level reading modes and performed per-voice sensitivity calibration to improve timing and synchronization with visual highlighting. This required iterative tuning to balance playback accuracy and responsiveness across different browser environments. Overall, his work established the core pipeline for converting extracted text into speech-ready units and enabled downstream features such as highlighting, click-to-read interaction, and AI-assisted processing. He also contributed to system integration, debugging, and final validation.

Xiaoxing (Sean)

Xiaoxing was primarily responsible for implementing the interactive reading experience in VIVIDpdf, including real-time highlighting, click-to-read functionality, skip-zone support, and playback-oriented content filtering. His work focused on making the reading process visually traceable, user-controllable, and adaptable to real-world PDF documents.

He designed and implemented a real-time highlighting system using a Canvas overlay, enabling precise token-level highlighting synchronized with audio playback while preserving the original document appearance. Hover-based sentence highlighting was also introduced to improve readability and interaction. A key technical contribution was cross-page highlighting, which consistently tracked and highlighted sentences spanning multiple pages across visible regions, improving continuity and user awareness.

Xiaoxing also developed the click-to-read feature, which allows playback to start from any selected word. This required accurate token hit detection, sentence reconstruction, and synchronization between visual and audio states. In addition, he implemented the Skip Zones feature, enabling users to exclude repeated layout elements such as headers, footers, and page numbers through customizable, per-page regions with dynamic editing support.

To further enhance reading quality, he introduced URL, email address, and bracketed content filters, along with customizable pronunciation rules that support multiple matching strategies and case sensitivity. Duplicate-rule detection and visual feedback were also incorporated. Overall, his work significantly improved the usability, flexibility, and interpretability of the reading experience and contributed to system integration, debugging, and final validation.

Joshua

Joshua was primarily responsible for developing the AI enhancement workflow, the persistence layer, the file management system, and several product-level support features for VIVIDpdf. His work focused on improving the system's intelligence, reliability, and long-term usability, extending it beyond a basic PDF reader into a more complete and persistent application.

He designed and implemented the AI enhancement pipeline, which captures screenshots of sentence regions, combines them with OCR-extracted text, and sends both to language models for correction. The system supports multiple models, including GPT-4o, GPT-4o-mini, Gemini 2.5 Flash, and Gemini 3 Flash Preview, and enables improved OCR accuracy, support for mathematical expressions, and speech-friendly formatting. User-defined AI instructions were also introduced to support customization across different document types and reading preferences.

To improve robustness and responsiveness, Joshua implemented prefetching for upcoming sentences, along with request queueing, concurrency control, and result caching. A multi-level fallback mechanism was developed to handle API failures, enabling automatic switching between models and clear error reporting. Additional features, such as real-time API key validation and persistent cost tracking, improved transparency and usability.

Joshua also implemented local persistence and file management using IndexedDB, supporting storage of recent files, thumbnails, reading state, and user settings. Features such as grid and list views, storage limits, automatic cleanup, and deletion confirmation were included. He also contributed onboarding guidance and built a bug-reporting tool that captures system information. Overall, his work significantly enhanced system robustness, scalability, and product quality, and supported integration, debugging, and final validation across the system.

Acknowledgements (Author: Joshua)

The successful realization of VIVIDpdf (Very Intelligent Vocal Interpreter for Digesting PDF) was made possible through the guidance of our project mentors and the rigorous standards of the ECE496 program. We extend our deepest gratitude to our supervisor, Professor Belinda Wang, whose emphasis on user-centric design was pivotal in defining the scope of our project. By challenging us to deeply investigate students' pain points with reading disabilities, she helped us prioritize high-impact features such as User-Defined Pronunciation over broader accessibility objectives. Furthermore, we are very grateful for having Professor Hamid Timorabadi as our project administrator. His feedback on our technical documentation and presentation, and the constructive guidance he provided, instilled in us a systematic approach to the professional engineering practice, from initial proposal to final validation. Our communication skills were also significantly sharpened by the engineering Communication Instructors, whose frequent consultations throughout the year were instrumental in refining our technical writing and presentation delivery.

The foundations of the open-source community and the generosity of our peers inspired our technical progress. Joel Purra's "Talkie" application, which served as a primary motivator for our design, heavily inspired our project. We also credit the developers of PDF-Extract-Kit and the OpenAI community for providing the essential libraries and APIs required for our implementation. In addition, we acknowledge Gemini and GPT's role in improving our development productivity and debugging workflows. We are also grateful to our friends and beta testers within the student community who provided the initial feedback necessary to refine the application's stability and performance.

Finally, we wish to acknowledge the invaluable internal support system shared by our team—Nathan Hung, Henry Li, Xiaoxing Liu, and Joshua Chau. Throughout a challenging academic year defined by high workloads and personal hurdles, the team maintained a culture of mutual understanding and flexibility. Whether stepping in for a teammate during illness or coordinating around competing deadlines, this spirit of collaboration was the cornerstone of our project's completion. Last but not least, we extend our thanks to all those who contributed to our journey in ways large and small; even if your names do not appear here, your support was vital to our success.

Executive Summary (Author: Sean)

VIVIDpdf (Very Intelligent Vocal Interpreter for Digesting PDF) is a browser-based, AI-enhanced reading system developed to improve text-to-speech access for academic and technical PDF documents. Conventional TTS tools often perform poorly on equations, citations, and repeated layout elements, making complex documents difficult to follow. VIVIDpdf addresses this problem by combining document parsing, interactive playback controls, and context-aware transcript enhancement into a practical reading tool.

The final system supports PDF upload and rendering, synchronized visual highlighting, selective playback from user-chosen locations, skip zones for non-prose regions, custom pronunciation rules, and filtering of unwanted text patterns. It also provides adjustable voice and speed settings, persistent local storage, and cross-platform browser-based access. For more difficult content, an AI enhancement mode improves OCR-derived text and transforms mathematical expressions into more natural, speech-ready output.

The completed design was evaluated through requirement-based testing on real academic documents. Results showed strong support for the project's core functional requirements, including element skipping, context-aware mathematical reading, user-defined pronunciation, custom skipping rules, selective playback, and playback controls. The system also demonstrated cross-platform compatibility, multi-browser support, adaptive interface behaviour, and stable memory usage during operation. Public beta testing and deployment audits further indicated that the system was usable in practice and performed strongly in accessibility and web performance evaluation.

Overall, VIVIDpdf met its core project goal of making complex PDF documents easier to read aloud and easier to follow visually. The final design demonstrates that a fully browser-based architecture can deliver a practical and extensible accessibility tool for technical reading. The current implementation is strongest on text-based PDFs, while future work includes broader support for scanned documents, further mobile refinement, and lower-cost approaches to AI-assisted transcript enhancement.

Table of Contents (Author: Joshua)

1.0 Introduction (Author: Henry)..... 14
1.1 Background & Motivation (Author: Henry)..... 14
1.2 Project Goal (Author: Henry)..... 14
1.3 System Requirements Specification (SyRS) (Author: Henry)..... 14
2.0 Final Design (Author: Joshua)..... 17
2.1 System Block Diagram (Author: Joshua)..... 17
2.2 System-level Overview (Author: Joshua)..... 17
2.3 Module-level Overview (Author: Joshua)..... 18
2.3.1 User Interface (UI) (Author: Joshua)..... 18
2.3.2 PDF Text Extraction (Author: Henry)..... 19
2.3.3 Transcript Generation (Author: Nathan)..... 20
2.3.4 Vocalizer (Author: Sean)..... 21
2.4 Assessment of final design (Author: Joshua)..... 22
3.0 Testing and Verification (Author: Sean)..... 23
3.1 Public release beta testing (Author: Nathan)..... 26
4.0 Summary and Conclusions (Author: Nathan)..... 26
4.1 Summary (Author: Nathan)..... 26
4.2 Conclusions (Author: Nathan)..... 26
4.3 Applications and Impact (Author: Nathan)..... 26
4.4 Future Work (Author: Nathan)..... 27
5.0 References (Author: Joshua)..... 28
6.0 Appendices (Author: Nathan)..... 31
Appendix A: Gantt Chart History..... 31
Appendix B: Financial Plan..... 34
Appendix C: Analysis of Technical Limitations in Existing Market Tools..... 37
Appendix R5: Selective Playback (User-Controlled Entry Points)..... 39
Appendix R8: Operating Systems Support..... 40
Appendix R10: Responsive UI / Adaptive Layout Support..... 41
Appendix R9: Non-Chromium Browsers Support..... 42
Appendix O: UX Click-to-photon Latency..... 43
Appendix T1: Element Skipping Verification..... 44
Appendix T2: Context-Aware Semantics Verification..... 45
Appendix T3: User-Defined Pronunciation Verification..... 47
Appendix T4: Custom Skipping Rules Verification..... 49
Appendix T5: Selective Playback Verification Test..... 52
Appendix T6: Web Performance and Accessibility Verification..... 53
Appendix T7: Playback Settings Verification..... 55
Appendix T8: Multi-OS Support Verification..... 57
Appendix T9: Multi-Browser Engine Support Verification..... 60
Appendix T11: Memory Integrity Verification..... 62

Appendix T12: Multi-Language Accessibility Verification..... 64
Appendix T13: Interaction Latency Verification..... 65
Appendix T14: Large File Reliability Verification..... 67
Appendix T15: Open-Source License Compliance Verification..... 68

1.0 Introduction (Author: Henry)

This report summarizes the motivation, design, implementation, and testing of VIVIDpdf, an AI-enhanced, context-aware text-to-speech (TTS) system specifically designed for academic documents. The report documents the project's growth from initial requirements to final verification, concluding with an assessment of the results and suggestions for future work.

1.1 Background & Motivation (Author: Henry)

Digital documents are central to modern higher education [1]. However, an estimated 3–7% of North American students have dyslexia, and many others struggle with reading [2]-[6]. For these students, reading dense documents is slow and tiring. Research shows that Text-to-Speech (TTS) tools help reduce the effort needed to read and improve understanding [7]-[9]. Despite a global TTS market worth billions of dollars [10], existing tools still handle technical and scientific text poorly.

We tested leading TTS platforms: Speechify, Microsoft Edge, and NaturalReader, on STEM documents and found consistent errors [11]-[12]. These tools misread math (e.g., reading $O(n^2)$ as "O N two"), mishandle chemical formulas, and fail to skip non-text elements, such as citations and page headers [13]. (See Appendix C for specific examples and a comparison table.) These errors produce broken, confusing audio that makes standard TTS tools unusable for engineering and science students [14].

VIVIDpdf addresses this problem with a content-aware, AI-assisted TTS system. It combines PDF layout analysis with Large Language Models (LLMs) to read technical notation correctly, skip non-prose elements, and let users define custom pronunciation rules, features that existing tools do not provide.

1.2 Project Goal (Author: Henry)

The goal of this project is to build an AI-enhanced, cross-platform TTS system tailored for structured academic and technical documents. The system aims to provide natural, coherent speech while granting users granular control through customizable features, such as user-defined pronunciation and custom skipping rules, to ensure high accuracy and address accessibility gaps.

1.3 System Requirements Specification (SyRS) (Author: Henry)

This subsection outlines the essential functional and non-functional requirements, objectives, and constraints that define our project.

Table I: System Requirements Specification.

ID	Specification	Description
R1	**Functionality:** Intelligent Element Skipping	**Requirement:** The system shall identify and provide an option to skip non-prose elements during playback, such as headers, footers, page numbers, and URLs.
R2	**Functionality:** Context-Aware Semantics	**Requirement:** The system shall interpret and read text based on its context. For example, correctly reading math expressions (reading "x^3" as "x cubed").
R3	**Functionality:** User-Defined Pronunciation	**Requirement:** The system shall allow users to create custom pronunciation rules for specific words, acronyms, or phrases (e.g., mapping "SQL" to "sequel").
R4	**Functionality:** Custom Skipping Rules	**Requirement:** The system shall allow users to create custom rules to ignore specified text patterns, such as content in brackets ([...]) or parentheses ((...)).
R5	**Functionality:** Selective Playback (Appendix R5)	**Requirement:** The system shall allow users to initiate playback from a specific page, section, or paragraph within the document.
R6	**Accessibility:** Web Performance and Compliance	**Requirement**: The system shall meet performance and accessibility standards from industry-standard auditing tools to ensure a smooth and inclusive user experience.
R7	**Functionality:** Playback Settings	**Requirement:** The system shall provide voice selection and speed control.
R8	**Compatibility**: Supporting ≥ 3 OS Platforms (Appendix R8)	**Requirement:** The system shall support at least three operating systems (Windows, macOS, and iPad OS) to ensure broad deployment.
R9	**Compatibility**: Supporting ≥ 3 Web Engines (Appendix R9)	**Requirement:** The system shall operate consistently across the three major browser engines (Blink, WebKit, and Gecko), covering > 90% of global users.
R10	**Accessibility**: Adaptive UI Display (Appendix R10)	**Requirement:** The system shall provide a responsive layout that adapts to desktops and tablets, maintaining usability and visual consistency across devices.

R11	**Performance**: Memory Integrity	**Requirement:** The system shall pass memory-profiling tests with zero detected leaks to ensure long-term stability and prevent performance degradation.
O1	**Accessibility**: Multi-Language Readability	**Objective:** The system should be able to read at least three languages.
O2	**Performance**: Interaction Latency < 96 ms (Appendix O)	**Objective:** The system should maintain click-to-photon latency below this threshold to maximize responsiveness and perceived interactivity. Ensuring responsive, smooth interaction by keeping visual feedback within the user's perceptual threshold for instantaneous action.
C1	**Reliability:** Large File Processing	**Constraint:** The system must open and process any file up to 1 GB in size.
C2	**Legal**: Open-Source License Compliance	**Constraint:** The system must follow OSI-approved open-source licenses for all dependencies. Prevents legal or licensing conflicts.
C3	**Cost**: Deployment Budget ≤ $250 CAD	**Constraint:** Total server and deployment costs must not exceed the given funding, ensuring the project remains within budget.

2.0 Final Design (Author: Joshua)

The following documents our final design. The system uses a four-stage pipeline consisting of the (1) User Interface, (2) PDF Text Extraction, (3) Transcript Generation, and (3) Vocalizer modules.

2.1 System Block Diagram (Author: Joshua)

Figure 1 shows the organization of four modules.

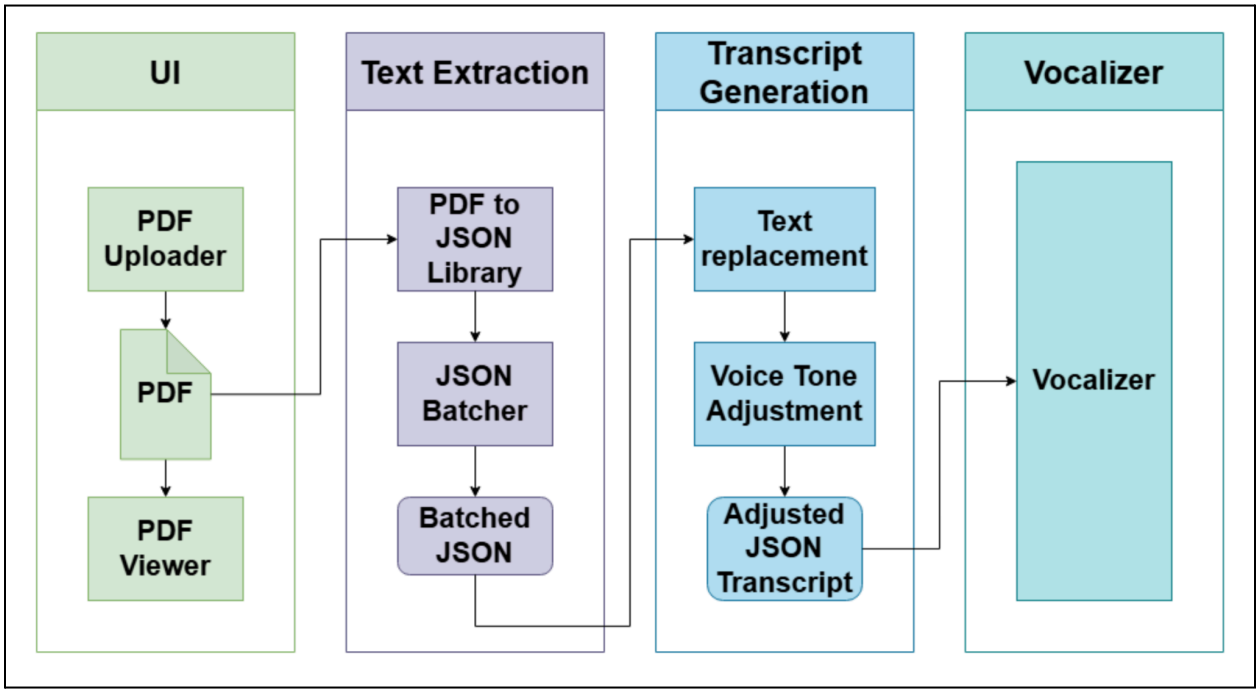


Figure 1. System Block Diagram of VIVIDpdf.

2.2 System-level Overview (Author: Joshua)

The system contains four modules: user interface, PDF text extraction, transcript generation, and vocalizer (Figure 1). Each module encapsulates a task and interacts with others only through well-defined interfaces. Such modularization improves maintainability, testing, and scalability [15]-[16]. This design also enables us to leverage existing, well-studied components for PDF parsing, text-to-speech vocalization, and PDF visualization [17]-[18].

The system takes in a PDF document and outputs the extracted text as audible speech through four modules:

1. The User Interface module accepts a PDF file from the user via file selection, drag-and-drop, or recent file history, and renders each page on a canvas.
2. The PDF Text Extraction module uses Mozilla's pdf.js library to parse each page and extract raw text items, including character strings, positions, and font metadata.
3. The Transcript Generation module merges fragmented tokens, groups them into sentences, and applies user-configured skipping rules and custom pronunciations. If AI Fix Mode is enabled, it sends each sentence with a cropped image to a multimodal LLM (Gemini or OpenAI) to correct notation and remove non-spoken elements.
4. The Vocalizer module speaks the processed text through the browser's built-in Web Speech API. It reports playback progress to the UI, highlighting the active word or sentence and auto-scrolling to keep the reading position in view.

2.3 Module-level Overview (Author: Joshua)

In our design, each module's well-defined inputs and outputs specify how it interacts with other modules. The four sections below detail each module's purpose, inputs, outputs, interactions with other system components, and key technical requirements.

2.3.1 User Interface (UI) (Author: Joshua)

The first module is the system's front end, built as a React web app. It supports PDF file input via the file selector, drag-and-drop, and reopening from the local history. It renders each PDF page using a canvas and an invisible text layer, and provides a bottom player bar with playback controls (play/pause, page jump), zoom, rotation, and dark mode. A settings panel allows users to select voice, language, speed, reading mode (word or sentence), and optionally enable AI Fix Mode. A speech customization panel lets users define skipping rules and custom pronunciations. Users can also draw skip zones directly on the page to exclude regions from playback. During playback, the UI highlights the active word or sentence and scrolls to keep up with the reading position. File metadata and reading progress are persisted locally using IndexedDB.

Inputs (from user):

- PDF file (via file picker, drag-and-drop, or reopen from history).
- Playback commands (play/pause, click on a word or sentence to jump).
- Settings (voice, language, speed, reading mode, AI Fix Mode toggle, highlighting options, zoom, rotation, dark mode, page navigation).
- Speech customization (skipping rules, custom pronunciation mappings).
- Skip zone drawings on the PDF page.

Input (from other modules):

- Processed token objects from the Transcript Generation module (used for highlighting and click-to-seek).
- Playback progress events from the Vocalizer (used to update the active highlight position and trigger auto-scrolling).

Outputs:

- The PDF File object to the PDF Text Extraction module.
- User preferences (voice, speed, skipping rules, custom pronunciations, skip zones, and AI configuration) to the Transcript Generation and Vocalizer modules.

Module Interactions:
The UI coordinates all other modules within the browser:

- It reads the user's PDF file as an ArrayBuffer and passes it to the PDF Text Extraction module for parsing.
- It renders each PDF page on a canvas with an invisible text layer overlay. As each page becomes visible, extracted tokens from the Transcript Generation module are registered for click and highlight interaction.
- When the user clicks play or clicks a word on the page, the UI passes the corresponding text and user settings to the Vocalizer, which speaks it through the browser's built-in speech engine. The UI then updates word/sentence highlighting and auto-scrolling based on playback progress events returned by the Vocalizer.

2.3.2 PDF Text Extraction (Author: Henry)

This second module is the “eye” of the system. It opens a PDF file, parses its contents, and extracts all raw text data. It uses Mozilla's PDF.js (pdfjs-dist) library, which runs entirely in the browser, to handle various PDF versions and encodings. The library's *getTextContent()* API extracts structured text items, including character strings, positional coordinates, and font metadata from each page. The module also performs scanned PDF detection by checking the first several pages for text content, and warns the user if the document appears to be image-only.

Input: a (browser) File object provided by the user through the file picker or drag-and-drop interface, which is read as an *ArrayBuffer* and passed to the pdf.js library for parsing.

Output: a *TextContent* object for each page, containing a flat array of text items. Each item includes the character string, a positional transformation matrix, and font metadata of the PDF.

Module Interactions:
The module reads PDF pages on demand and feeds raw text:

1. When the user uploads or reopens a PDF, the UI Module reads the file and passes it to pdf.js, which loads the document structure. Text extraction then occurs lazily on a per-page basis. Each page's text is extracted only when that page becomes visible on screen, rather than processing the entire document up front.
2. Once a page's text is extracted, the resulting text items are passed directly to the next Transcript Generation module, and the final processed tokens are registered with the UI Module for playback and interaction.

2.3.3 Transcript Generation (Author: Nathan)

This third module is the core "brain" of the system. It transforms raw extracted text into a cleaned transcript. It operates in three stages. First, a token-merging pass reassembles words that may have been fragmented across multiple text spans during PDF extraction using spatial heuristics. Second, a sentence boundary detection algorithm groups the merged tokens into sentences based on punctuation, font changes, and paragraph spacing. Third, users may optionally enable AI Fix Mode, which sends each sentence along with a cropped image of its visual context to a multimodal LLM to correct OCR artifacts and interpret mathematical notation into spoken form (e.g., "x^2" → "x squared"). At playback time, user-configured skipping rules and custom pronunciation mappings are applied to produce the final spoken transcript.

Inputs: an array of raw token objects from the PDF Text Extraction stage. Additionally, it receives user-configured settings from the UI Module, including toggle-based skipping rules (e.g., skip URLs, emails, bracketed text), custom pronunciation mappings, user-drawn skip zones, and, when AI Fix Mode is enabled, a cropped image of each sentence for visual verification by the LLM.

Output: an array of processed token objects grouped into sentences. Each token retains its spoken text, position on the page, and font metadata.

Module Interactions:
This module processes raw text into transcripts and passes them to the vocalizer:

1. It receives raw text token objects from the PDF Text Extraction module as each page is rendered.
2. It receives user settings (skipping rules, custom pronunciations, skip zones) from the UI Module and applies them to filter and modify the spoken text at playback time.
3. When AI Fix Mode is enabled, it sends each sentence's raw text and a cropped image to an external LLM API (Gemini or OpenAI) and caches the corrected transcript for reuse.
4. The processed token objects are registered with the UI Module for highlighting and click-to-seek, and their spoken text is passed to the Vocalizer for speech synthesis.

2.3.4 Vocalizer (Author: Sean)

This fourth module is the "voice" of the system. It uses the browser's built-in Web Speech API (*SpeechSynthesis*) to convert the transcript into natural-sounding audio, giving users access to a wide range of voices from multiple providers, including online cloud-based voices (e.g., Google) and offline local voices (e.g., Microsoft, Apple). Users can select their preferred language and voice, with the system clearly labelling each option as either Offline or Requires Internet. To ensure uninterrupted playback, the vocalizer includes a configurable offline fallback mechanism. If the selected online voice engine fails to respond, the system switches to a user-specified local voice with its own independent speed setting.

Inputs: the transcript text to be spoken, and user preferences from the UI Module including the selected voice (e.g., "Microsoft Aria," "Google US English"), language (e.g., English, French), speed (0.5x–3.0x), speech customization rules (e.g., skip URLs, emails, bracketed text), and custom pronunciation mappings (e.g., "IEEE" → "I triple E").

Output: synthesized speech audio played directly in the user's browser via the Web Speech API.

Module Interactions:

The vocalizer converts text to speech and playback back to the UI:

1. The vocalizer is invoked by the UI Module when the user initiates or continues playback (via the play button or by clicking a word/sentence on the page).
2. It operates in one of two reading modes: Word Mode, which batches multiple sentences into a single utterance and uses the (*onboundary*) event to track word-level progress for per-word highlighting; or Sentence Mode, which processes one sentence at a time and highlights the entire sentence during playback. (Note: Word Mode is only available with voices that support the (*onboundary*) event, such as Microsoft and Apple voices; Google online voices are limited to Sentence Mode.)
3. The browser's *SpeechSynthesis* engine handles audio generation and playback directly. Instead, the vocalizer communicates playback progress back to the UI through event callbacks (*onstart*, *onboundary*, *onend*), which drive the active word/sentence highlighting and auto-scrolling.
4. If AI Fix Mode is enabled, the vocalizer first sends each sentence to an AI service (Gemini or OpenAI) for transcript correction before speaking, and pre-caches corrected sentences ahead of the reading position to minimize gaps.

2.4 Assessment of final design (Author: Joshua)

The final design of VIVIDpdf successfully delivers an AI-enhanced, context-aware TTS solution specifically engineered for complex academic documents. The system successfully addressed the gap in commercial TTS tools, which inaccurately read technical content and lacked granular user control, by meeting all requirements and objectives in SyRS, as shown in section 5.0.

The system's major strengths stem from its modular architecture and targeted engineering choices that provided both robustness and specialized functionality.

- Modular Architecture: The four-stage pipeline (UI, PDF Text Extraction, Transcript Generation, Vocalizer) proved highly effective, enabling different team members to develop and debug modules independently, thereby simplifying integration.
- Robustness: The design includes substantial engineering efforts to ensure robustness and reliability. We implement request queueing, prefetching, result caching, and a multi-level fallback mechanism for the LLM pipeline. This resilience is crucial for a system dependent on external API services, improving responsiveness and cost transparency through real-time tracking.
- Vocalizer Trade-off: A core design trade-off in the Vocalizer module involved prioritizing the use of the browser's native Web Speech API (SpeechSynthesis) for cross-platform compatibility and reliability. This decision means that word-level highlighting (Word Mode) is only available with voices that support the onboundary event (typically Microsoft and Apple voices), while Google online voices are restricted to sentence-level highlighting. This was a necessary trade-off to avoid the complexity and cost of a proprietary audio streaming solution while retaining broad voice selection (R7).

<u>Limitations and Future Improvements</u>

Despite its success, the final design has limitations based on initial constraints and technological boundaries, which outline clear paths for future work.

For Cost Management (C3), the reliance on external, pay-per-use APIs to meet the Context-Aware Semantics (R2) requirement means the project operates under a tight deployment budget. Future work should explore more efficient local models or more aggressive caching strategies to reduce reliance on external API calls and maintain long-term cost-effectiveness.

3.0 Testing and Verification (Author: Sean)

Table II lists the testing procedures and results used to validate the system's requirements. To test under realistic conditions, a variety of academic documents were tested. For example, calculus textbooks were used to test equation-reading, and heavily formatted research papers were used to test element skipping, custom pronunciation rules, and in-text citation skipping. The system successfully passed all functional and non-functional evaluations.

Table II: Project Verification Table

ID	Project Requirement	Verification Method	Verification Result and Proof
R1	**Element Skipping:** Users can skip elements such as headers, footers, and page numbers.	**TEST**: Read a PDF with headers and footers. Check if marked items are skipped.	**PASS.** System omitted all marked headers and footers. See Appendix T1 for screenshots.
R2	**Context-Aware Semantics:** correctly read expressions based on semantic context (e.g., read "x^3" as "x cubed").	**TEST:** Read a PDF containing mathematical expressions. Compare spoken output to expected verbal expressions.	**PASS.** The system correctly articulated expressions such as "x cubed". See Appendix T2 for sample text and audio transcript.
R3	**User-Defined Pronunciation:** Users can create and apply custom pronunciation rules.	**TEST:** Add pronunciation rules (e.g., "SQL → sequel") and verify correct playback. Ensure persistence after reload.	**PASS.** Playback matched the user-defined pronunciations. See Appendix T3 for GUI configuration screenshots and audio transcript.
R4	**Custom Skipping Rules:** Users can define text patterns to be ignored during playback.	**TEST:** Define rules (e.g., ignore text, URL, in parentheses) and confirm that playback excludes those sections.	**PASS.** The system skipped defined patterns. See Appendix T4 for GUI configuration screenshots and audio transcript.
R5	**Selective Playback**: Playback can start from any paragraph, sentence, or word.	**TEST:** The system shall allow users to initiate playback from a specific page, section, or paragraph within the document.	**PASS.** Playback initiated exactly at the requested index (clicked by the user). See Appendix T5 for screenshots and audio transcript.

R6	**Web Performance and Compliance**: meeting performance and accessibility standards from industry standard auditing tools	**TEST:** Execute a Google PageSpeed Insights desktop analysis on the production URL.	**PASS.** The application achieved a Performance score of 99 and an Accessibility score of 100. See Appendix T6 for PageSpeed Insights screenshots.
R7	**Playback Settings:** Support voice selection and adjustable speed control.	**TEST:** Modify voice selection and playback speed. Verify that audio output updates correctly during playback (without crashes, playback failure, or UI malfunction).	**PASS**. The system allowed voice selection and real-time speed adjustment. See Appendix T7 for screenshots.
R8	**Multi-OS Support:** Supporting ≥ 3 OS Platforms	**TEST:** Run the system on three different operating systems. Verify consistent behaviour in document upload, rendering, parsing, and playback.	**PASS**. The system functioned correctly across all tested OS platforms with consistent behaviour. See Appendix T8 for screenshots.
R9	**Multi-Browser Engine Support:** Supporting ≥ 3 Web Engines.	**TEST**: Run the system on Chrome, Safari, and Firefox. Verify correct rendering, functionality, and audio playback.	**PASS**. The system operated correctly across all tested browsers. Rendering, parsing, and playback were consistent. See Appendix T9 for screenshots.
R10	**Adaptive UI Display**: Responsive layout adapts to desktops, tablets, and smartphones.	**TEST**: Inspect layout at multiple screen sizes. Verify UI components remain visible, aligned, and functional.	**PASS**. Interface displayed correctly across all tested screen sizes with consistent layout and functionality. See Appendix T8 for screenshots.
R11	**Memory Integrity:** The system shall pass memory profiling with zero leaks.	**TEST:** Monitor app data usage across three states: initial load, after storing multiple large PDFs, and after executing the in-app deletion command.	**PASS**. Storage usage returned to baseline after file deletion. See Appendix T11 for storage monitoring screenshots.

O1	**Accessibility**: The system must read text in at least three languages.	**TEST**: Upload documents containing English, French, and Chinese. Verify the vocalizer correctly speaks each language.	**PASS**. The system correctly reads text in all three tested languages. (Appendix T12)
O2	**Performance**: Interaction Latency < 96 ms (Appendix O)	**TEST**: Measure Interaction to Next Paint (INP) via Chrome DevTools Performance Testing.	**PASS**. INP measured 80 ms. See Appendix T13 for DevTools traces.
C1	**Reliability**: The system must open and process any file up to 1 GB in size.	**TEST**: Upload a PDF document exceeding the 1 GB threshold to stress-test parsing and rendering engines.	**PASS**. The 1.84GB file loaded successfully. Playback functions normally. (Appendix T14)
C2	**Legal**: Open-Source License Compliance	**ANALYSIS:** Review the software license of all software libraries used.	**PASS**. All dependencies verified as OSI-approved. (Appendix T15)
C3	**Cost**: Deployment Budget ≤ $250 CAD	**ANALYSIS:** Review project financial records (Appendix B)	**PASS**. Deployment cost is $0. Development and project management costs total $158.91 CAD.

3.1 Public release beta testing (Author: Nathan)

On top of the team's efforts in testing our app, we also released VIVIDpdf publicly and received one feature request (superscript skipping) and two bug reports. In just three days, VIVIDpdf has served more than 200 users (Figure 2).

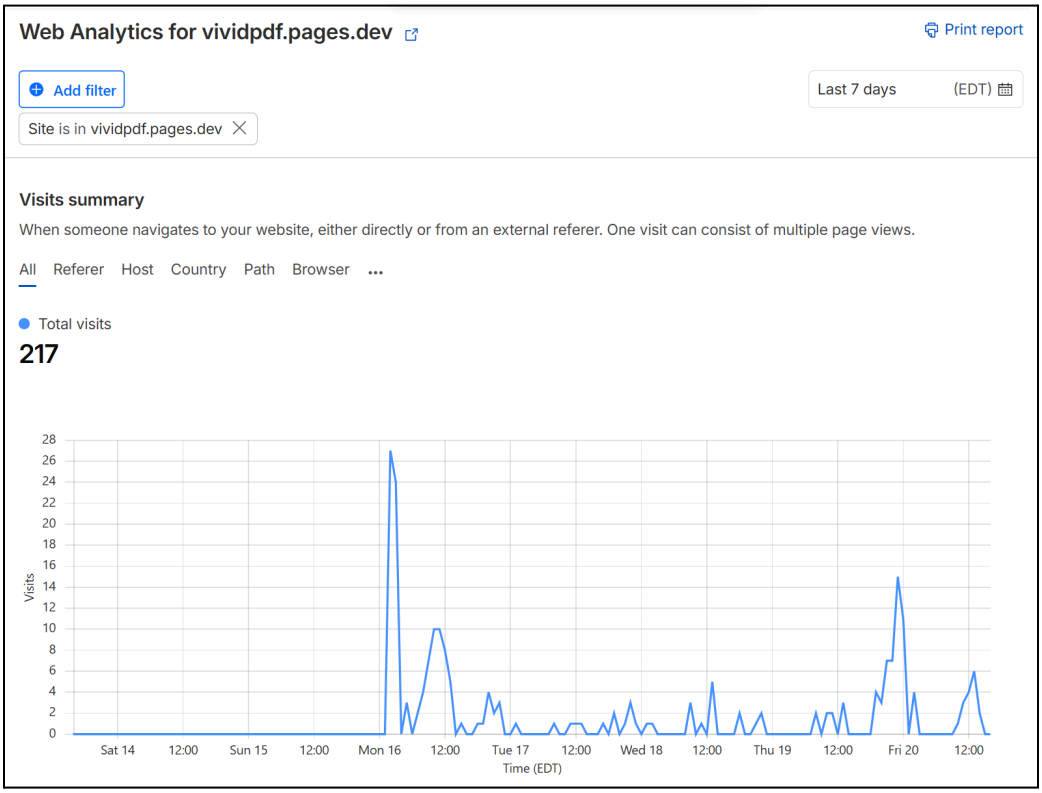


Figure 2. Statistics of VIVIDpdf public beta test user count.

4.0 Summary and Conclusions (Author: Nathan)

4.1 Summary (Author: Nathan)

We built VIVIDpdf, a browser-based text-to-speech tool for academic PDFs. It reads math equations, skips formatting like headers, and uses custom pronunciation rules. As demonstrated in Section 3.0, the system met all 11 mandatory requirements, 2 objectives, and 3 constraints.

4.2 Conclusions (Author: Nathan)

Our hypothesis that a fully client-side web app that reads PDF with all requirements is confirmed. The system processes files locally in the browser without a backend server. The four-stage modular design allowed independent development and easy integration.

4.3 Applications and Impact (Author: Nathan)

VIVIDpdf started as Nathan's personal project. After completing the first prototype six months ago, he has used the app continuously for his own studies. It has fully replaced Speechify, the

paid app he previously relied on. Over the past semesters, he has spent more than 100 hours listening to textbooks, handouts, and course readings. VIVIDpdf also proved a great aid in sleeping. He often listens to PDFs in bed and falls asleep while using the app. In fact, he is so satisfied by the project and so thankful for his teammate's work on a capstone that benefits him directly, he would often smile in dreams.

VIVIDpdf is also useful for anyone who wants to read academic PDFs aloud. The primary users are students and researchers in STEM fields, whose documents contain notation, equations, and dense formatting that standard TTS tools misread. It is also an accessibility tool for users with visual impairments or reading difficulties as it provides word-level highlighting and configurable reading speed. Because the system runs entirely in the browser, with no account, server, or data upload, it can be used in privacy-sensitive settings without IT approval or data-handling concerns. The open-source license allows tech-savvy students to modify the source code.

4.4 Future Work (Author: Nathan)

Our app meets our proposed requirements, but has clear areas for improvements. The following are some potential improvements and additional feature:

- **OCR for scanned PDFs**: The current system only supports text-based PDFs. Adding an in-browser OCR engine (e.g., Tesseract.js) would allow it to handle scanned documents and photos of printed text.
- **Local LLM models**: AI Fix Mode currently requires a paid API key for Gemini or GPT. Running a small language model locally in the browser (e.g., via WebLLM or ONNX Runtime Web) would remove this dependency on cloud costs and enable offline context-aware semantics.
- **Local in-browser voice generation models:** The application relies on operating system voices through the Web Speech API, causing inconsistent audio across devices. Implementing local voice generation models provides standard audio. This ensures users hear the exact same voices even on devices lacking pre-installed text-to-speech software.

5.0 References (Author: Joshua)

[1] G. O 'Su'illeabha'in, D. Lanclos, and T. Farrelly, Eds., How to Use Digital Learning with Confidence and Creativity : A Practical Introduction, First edition. Cheltenham, England: Edward Elgar Publishing Limited, 2024.

[2] R. K. Wagner et al., "The Prevalence of Dyslexia: A New Approach to Its Estimation," Journal of learning disabilities, vol. 53, no. 5, pp. 354–365, 2020, doi: 10.1177/0022219420920377.

[3] L. Eriksson, "Difficulties in academic reading for EFL students: An initial investigation," Language Teaching, vol. 56, no. 1, pp. 149–152, 2023. doi:10.1017/S0261444822000246

[4] H. Beidas, A. Khateb, and Z. Breznitz, "The cognitive profile of adult dyslexics and its relation to their reading abilities," Reading & writing, vol. 26, no. 9, pp. 1487–1515, 2013, doi: 10.1007/s11145-013-9428-5.

[5] A. Dębska et al., "The cognitive basis of dyslexia in school‐aged children: A multiple case study in a transparent orthography," Developmental science, vol. 25, no. 2, pp. e13173-n/a, 2022, doi: 10.1111/desc.13173.

[6] M. López-Zamora, N. Porcar-Gozalbo, I. López-Chicheri García, and A. Cano-Villagrasa, "Linguistic and Cognitive Abilities in Children with Dyslexia: A Comparative Analysis," European journal of investigation in health, psychology and education, vol. 15, no. 3, p. 37, 2025, doi: 10.3390/ejihpe15030037.

[7] S. Raffoul and L. Jaber, "Text-to-Speech Software and Reading Comprehension: The Impact for Students with Learning Disabilities," Canadian journal of learning and technology, vol. 49, no. 2, pp. 1–18, 2023, doi: 10.21432/cjlt28296.

[8] V. Clinton-Lisell, "Does Reading while Listening to Text Improve Comprehension Compared to Reading Only? A Systematic Review and Meta-Analysis Author Notes I thank Sofia Kafami and Yusuf Marafa for their assistance with screening and coding studies." Available: https://files.eric.ed.gov/fulltext/EJ1403866.pdf

[9] M. C. Young, "The Effects of Text-to-Speech on Reading Comprehension of Students with Learning Disabilities," ProQuest Dissertations & Theses, 2017.

[10] ltd, Research and Markets. Text-to-Speech - Global Strategic Business Report. https://www.researchandmarkets.com/reports/5303572/text-to-speech-global-strategic-business-report. Accessed 21 Mar. 2026.

[11] Chrome Web Store, “Text to Speech — Chrome Web Store,” [Online]. Available: https://chromewebstore.google.com/search/text%20to%20speech. Accessed Mar. 21, 2026.

[12] Text to Speech - Apple. https://www.apple.com/us/search/text-to-speech?src=globalnav. Accessed Mar 21. 2026.

[13] K. Kuligowska, P. Kisielewicz, and A. Włodarz, “Speech synthesis systems: disadvantages and limitations,” International Journal of Engineering & Technology, vol. 7, no. 2.28, p. 234, May 2018, doi: https://doi.org/10.14419/ijet.v7i2.28.12933.

[14] C. Clausner, S. Pletschacher, and A. Antonacopoulos, “The Significance of Reading Order in Document Recognition and Its Evaluation,” in Proceedings of the International Conference on Document Analysis and Recognition, IEEE, 2013, pp. 688–692. doi: 10.1109/ICDAR.2013.141.

[15] F. Beck and S. Diehl, “On the Congruence of Modularity and Code Coupling.” [Online]. Available: https://memphis-cs.github.io/comp-7085-8085-2012-fall/papers/Beck2011FSE.pdf?utm_source=chatgpt.com. Accessed: Mar 21, 2026.

[16] P. C. Jha, V. Bali, S. Narula, and M. Kalra, “Optimal component selection based on cohesion & coupling for component based software system under build-or-buy scheme,” Journal of computational science, vol. 5, no. 2, pp. 233–242, 2014, doi: 10.1016/j.jocs.2013.07.003.

[17] R. W. Sproat and J. P. Olive, “A Modular Architecture for Multilingual Text-to-Speech,” Progress in Speech Synthesis, pp. 565–573, 1997, doi: https://doi.org/10.1007/978-1-4612-1894-4_45.

[18] “PDF.js,” mozilla.github.io. https://mozilla.github.io/pdf.js/

[19] “Speechify Pricing: Free & Premium.” Speechify, https://speechify.com/pricing/. Accessed Mar 21 2026.

[20] Read Aloud. https://www.microsoft.com/en-us/edge/features/read-aloud. Accessed Mar 21 2026.

[21] *Plans & Pricing (Personal Version) | NaturalReader Help Center*. https://help.naturalreaders.com/en/articles/8854700-plans-pricing-personal-version. Accessed Mar 21 2026.

[22] H. Aldarmaki, A. Ullah, S. Ram, and N. Zaki, “Unsupervised Automatic Speech Recognition: A review,” *Speech communication*, vol. 139, pp. 76–91, 2022, doi: 10.1016/j.specom.2022.02.005.

[23] M. Tauseef, S. G. Naik, and K. S. Harsha, "SpeakEasy Reader: An OCR-Based Text-to-Speech Device for Enhanced Accessibility in Assisting Reading Challenges," in International Conference on Computing, Communication, and Networking Technologies (Online), IEEE, 2024, pp. 1–6. doi: 10.1109/ICCCNT61001.2024.10723858.

[24] S. Raffoul and L. Jaber, "Text-to-Speech Software and Reading Comprehension: The Impact for Students with Learning Disabilities," *Canadian journal of learning and technology*, vol. 49, no. 2, pp. 1–18, 2023, doi: 10.21432/cjlt28296.

[25] StatCounter GlobalStats, Desktop Operating System Market Share Worldwide, 2025. [Online]. Available: https://gs.statcounter.com/os-market-share/desktop/worldwide

[26] Wikipedia, Usage Share of Operating Systems, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Usage_share_of_operating_systems

[27] "Responsive Web Design (RWD) and User Experience." *Nielsen Norman Group*, https://www.nngroup.com/articles/responsive-web-design-definition/. Accessed Mar 21 2026.

[28] Parlakkiliç, Alaattin. "Evaluating the Effects of Responsive Design on the Usability of Academic Websites in the Pandemic." *Education and Information Technologies*, vol. 27, no. 1, 2022, pp. 1307–22. *PubMed Central*, https://doi.org/10.1007/s10639-021-10650-9.

[29] *150+ UX (User Experience) Statistics and Trends (Updated for 2025)*. https://userguiding.com/blog/ux-statistics-trends. Accessed 18 Oct. 2025.

[30] Wikipedia, Usage Share of Web Browsers – Firefox (Gecko), 2025. [Online]. Available: https://en.wikipedia.org/wiki/Usage_share_of_web_browsers

[31] LaViola, Joseph J. "A Discussion of Cybersickness in Virtual Environments." *ACM SIGCHI Bulletin*, vol. 32, no. 1, Jan. 2000, pp. 47–56. *DOI.org (Crossref)*, https://doi.org/10.1145/333329.333344.

[32] MacKenzie, I. Scott, and Colin Ware. "Lag as a Determinant of Human Performance in Interactive Systems." *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems - CHI '93* [Amsterdam, The Netherlands], 1993, pp. 488–93. *DOI.org (Crossref)*, https://doi.org/10.1145/169059.169431.

[33] Harrison, Chris, et al. "Rethinking the Progress Bar." *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology* [Newport Rhode Island USA], 2007, pp. 115–18. *DOI.org (Crossref)*, https://doi.org/10.1145/1294211.1294231.

6.0 Appendices (Author: Nathan)

Appendix A: Gantt Chart History

Link to access the complete Gantt Chart:
https://app.clickup.com/9017251686/v/li/901705849663
(Please kindly inform me if there is any access issue.)

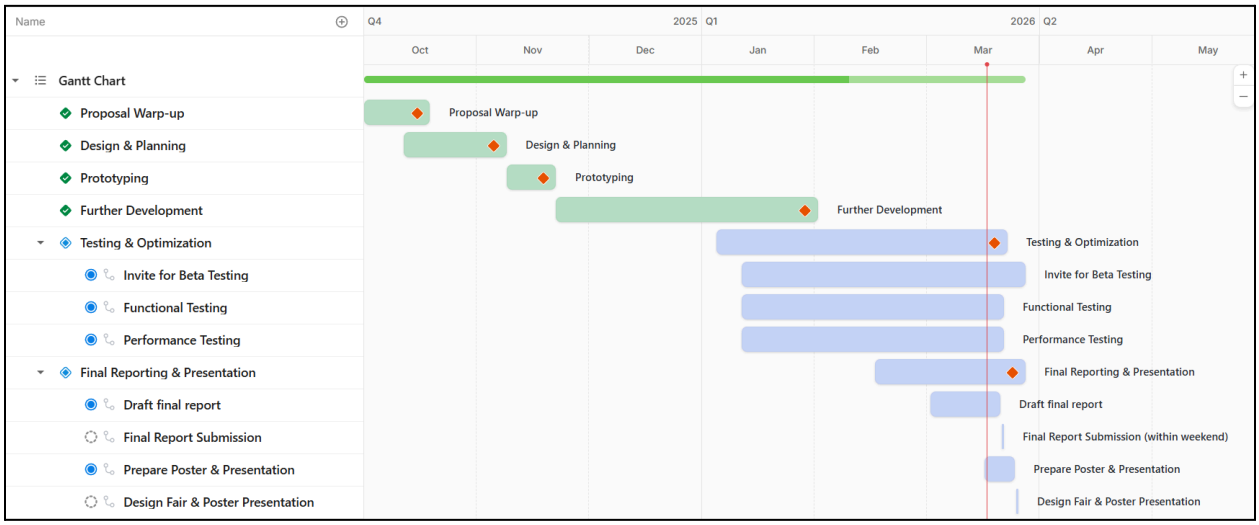


Figure 3. The final version of the entire Gantt Chart.

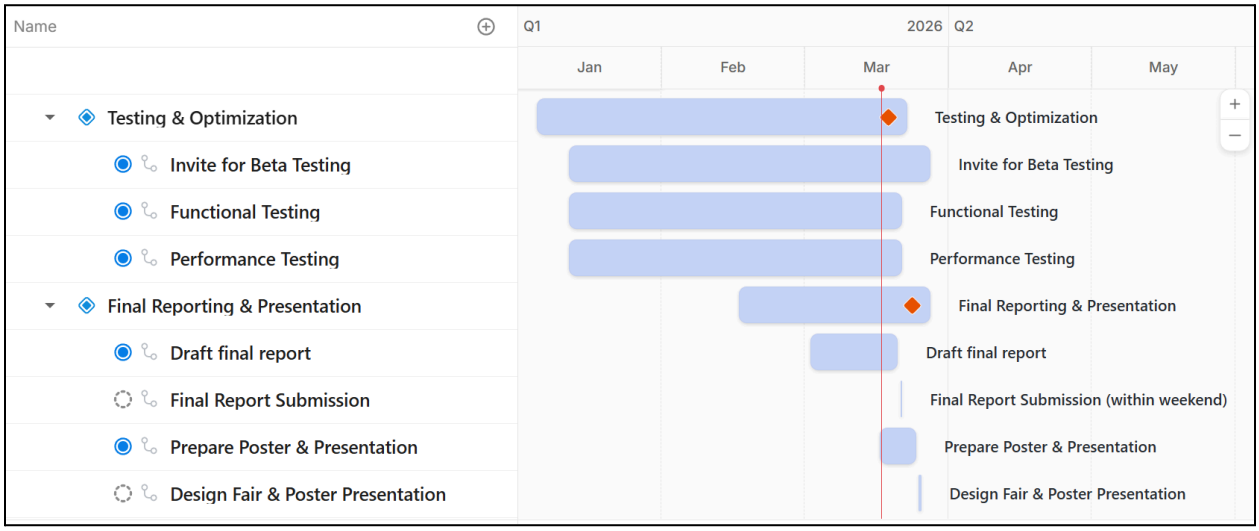


Figure 4. Zoomed-in view of the final version of the Gantt Chart.

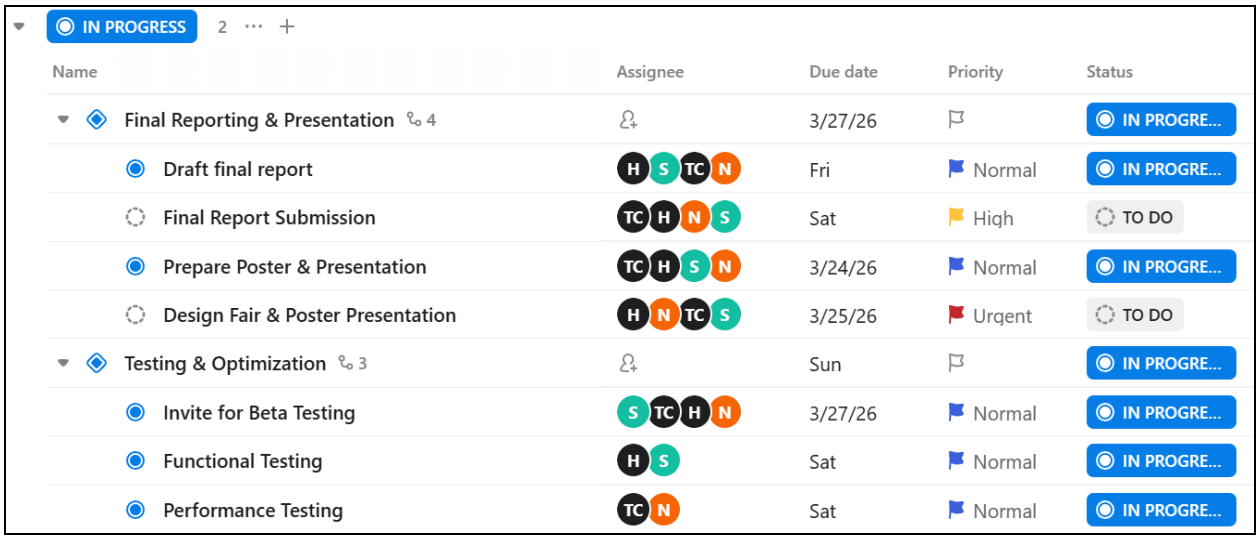


Figure 5. Detailed list of the remaining project tasks.

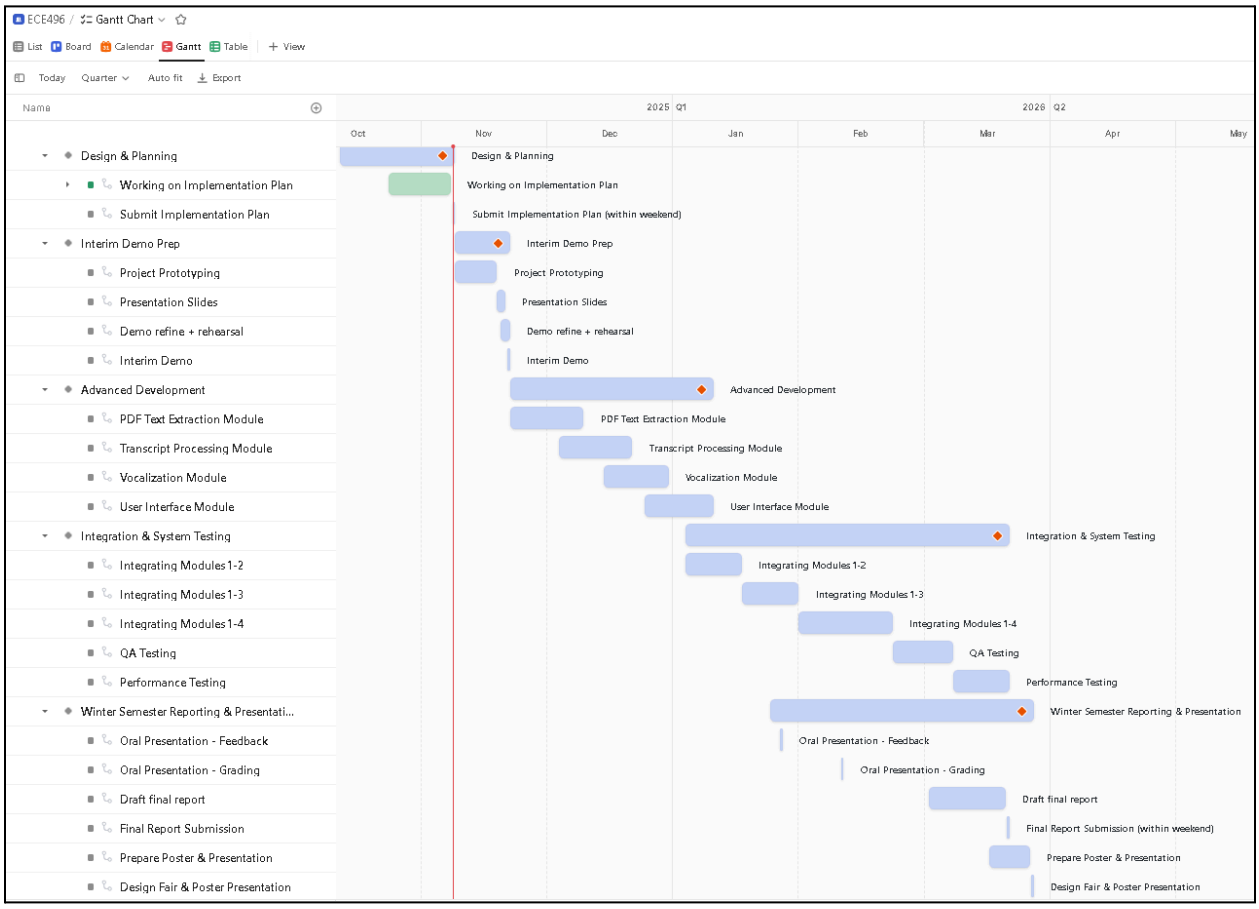


Figure 6. Past version of the Implementation Plan Gantt Chart.

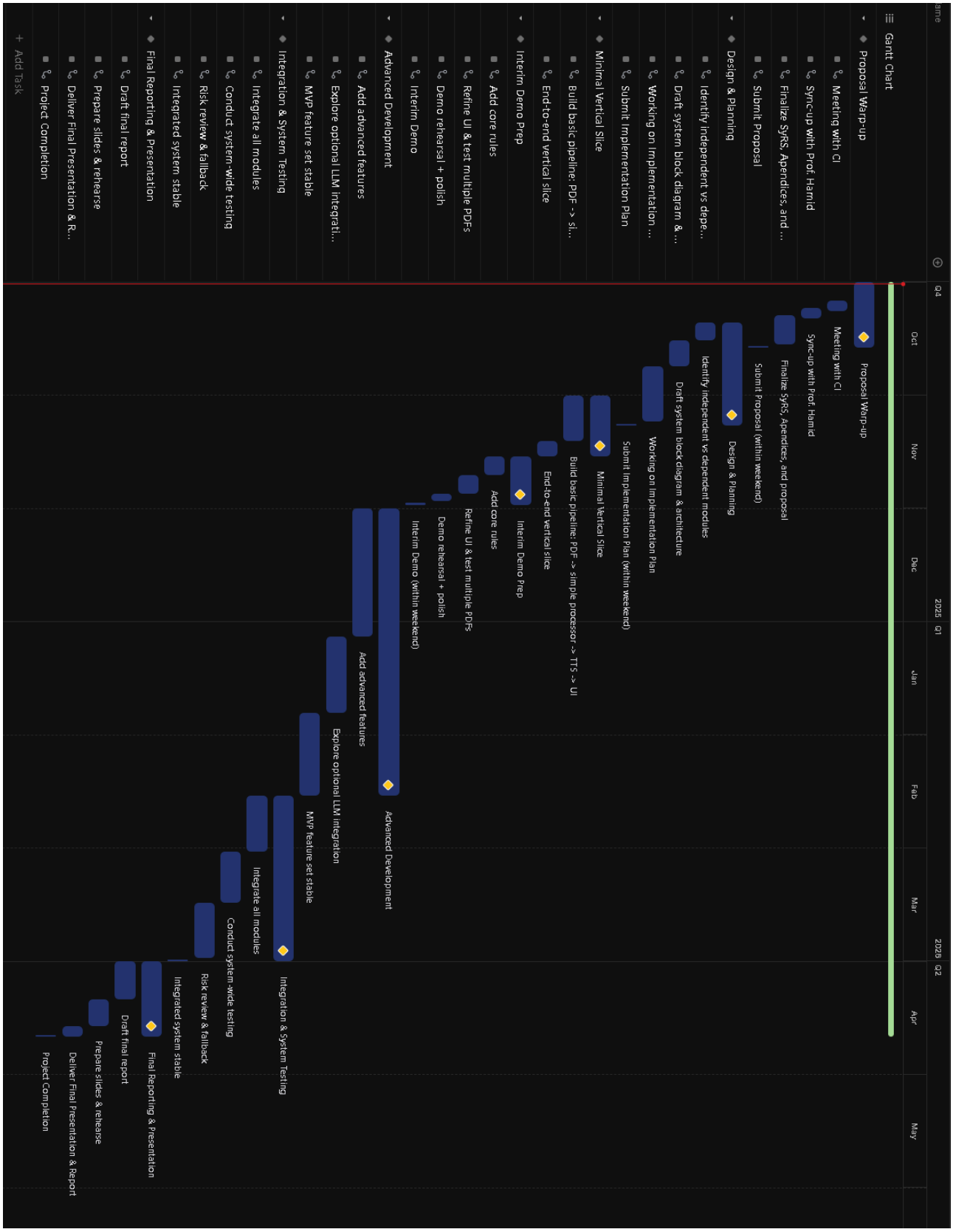


Figure 7. Past version of the Project Proposal Gantt Chart.

Appendix B: Financial Plan

Link to access the complete Budget Table: [Financial Plan - Budget Table]

Link to access the receipts of the expenses:
https://drive.google.com/drive/folders/1JVRAZtCyhdyX19DBSet38nLnYutFThv-?usp=sharing

ECE496 **Financial Plan** **Project Information**				
Project ID #:	**Project Title:**	**Students:**	**Supervisor:**	**Administrator:**
2025188	VIVID PDF: Very Intelligent Vocal Interpreter for Digesting PDF	Hanlin Li	Prof. Belinda Wang	Prof. Hamid Timorabadi
		Tsz Tung (Joshua) Chau		
		Nathan Hung		
		Xiaoxing (Sean) Liu		

Figure 8. The title of the final updated Budget Table.

Expenses							
	Consumables/Materials						
	Item #	**Description**	**Cost per item ($)**	**Quantity**	**Total Cost ($)**	**Purpose**	
	1	Cloud Hosting Service	0	1	0	small cloud instance + storage	
	2	API Usage	$ 9.75	1	$ 9.75	modest API calls for AI/TTS or OCR testing	
	3	Software Tools	$ 49.72	3	$ 149.16	open-source tools mostly free; allocate $50 for licenses or plugins	
	Sub-total				158.91		
	Capital Equipment						
	Item #	**Description**	**Cost per item ($)**	**Quantity**	**Total Cost ($)**	**Purpose**	**Notes**
	1	Hanlin's personal laptop	1000	1	1000	Research and work on course and project deliverables	We volunteer to pay ourselves
	2	Nathan's personal laptop	1000	1	1000		
	3	Sean's personal laptop	1000	1	1000		
	4	Joshua's personal laptop	1000	1	1000		
	Sub-total				4000		
	Student Labour						
	Item #	**Description**	**Cost per hour ($/hr)**	**Quantity (# of hours)**	**Total Cost ($)**	**Purpose**	**Notes**
	1	Hanlin Li	50	240	12000	To Appreciate how much time we have spent to deliver the project	We volunteer to pay ourselves
	2	Nathan Hung	50	240	12000		
	3	Sean Liu	50	240	12000		
	4	Joshua Chau	50	240	12000		
	Sub-total				48000		
	Total Cost of Project ($)				52158.91		
	Team Covers Ourselves ($)				-52000		
	Total Funding Requested ($)				**158.91**		

Figure 9. The expenses of the final updated Budget Table.

Funding				
	Students			$250
	Supervisor			$0
	Administrator			$0
	Extra Funding Requested from ECE496			$0
	Total Funding Receiving			**$250**

Figure 10. The funding of the final updated Budget Table.

What	Spent (USD)	In CAD	When	By
OpenAI API usage credit	$ 6.78	$ 9.75	11/1/2025 16:55:00	Nathan
Project Management Tool: ClickUp	$ 36.16	$ 49.72	1/1/2025	Joshua
Project Management Tool: ClickUp	$ 36.16	$ 49.72	2/1/2025	Joshua
Project Management Tool: ClickUp	$ 36.16	$ 49.72	3/1/2025	Joshua
		$ 149.16		
Funding As Discussed	$ 250.00			
Expenses in CAD based on receipts	$ 158.91			
Remaining	$ 91.09			

Figure 11. The requested funding calculation of the final updated Budget Table.

Appendix C: Analysis of Technical Limitations in Existing Market Tools

This appendix provides the technical evidence and comparative data that motivated the development of VIVIDpdf. Our research into the current Text-to-Speech (TTS) market highlights a significant "accessibility gap" in the processing of technical and academic STEM documents.

C.1 Feature Comparison of Leading TTS Software

As of October 2025, our market analysis of the most popular TTS platforms—Speechify [19], Microsoft Edge [20], and NaturalReader [21]—found that while these tools excel at narrative text, they lack the granular controls required for technical reading. Table III summarizes these findings:

Table III: Feature Support Comparison (Tested Oct 10, 2025).

Features	**Speechify**	**Microsoft Edge**	**NaturalReader**
Pricing	$29 USD/mo [19]	Free [20]	$20.9 USD/mo [21]
Skipping page elements (headers, footers, page numbers)	Yes [19]	No	Yes [21]
User-defined pronunciation	No	No	Yes [21]
Reading math syntax	No	No	No
User-defined element skipping	No	No	No
Skipping in-line elements (footers, references, URLs)	No	No	No

C.2 Empirical Evidence of Technical Pronunciation Failures

To measure the impact of these missing features, we conducted a "stress test" using standard technical terms found in academic engineering papers. The results in Table IV demonstrate how existing tools often produce nonsensical audio when encountering mathematical or symbolic notation.

Table IV: Sample Technical Text Pronunciation Failures.

sample text	**Pronunciation by Speechify**	**Correct (desired) pronunciation(s)**
runtime of $O(n^2)$	runtime of O N two	runtime of O of N square
for all e $\in \mathscr{X}$	for all E set membership symbol	for all E belonging to X
		for all E in X
NO_3 pollution	no three pollution	nitrate pollution
		N O three pollution
$c = 3 \cdot 10^8$ m/s	C equals three one hundred and eight M slash S	C equals three times ten to the eight M over S
		C equals three times ten to the eight meters per second
Prof. Hamid is awesome and Prof. Wang too	professor hamid is awesome six and professor wang too seven	professor hamid is awesome, source six, and professor wang too, source seven

C.3 Conclusion on Market Limitations

The evidence gathered confirms that leading TTS solutions are optimized for narrative prose rather than the symbolic and multi-modal nature of technical literature. The consistent failure to accurately read mathematical syntax [13] and the inability to follow complex multi-column reading orders [14] create a fragmented listening experience. This technological gap significantly increases cognitive load for students with reading challenges, necessitating the development of a domain-aware solution like VIVIDpdf.

Appendix R5: Selective Playback (User-Controlled Entry Points)

Allowing users to begin playback from a specific page, section, or paragraph aligns with established accessibility and usability principles in digital reading systems. Research in assistive technology consistently shows that fine-grained navigation (e.g., section- or paragraph-level playback) significantly improves comprehension and task efficiency for readers with visual or learning disabilities by reducing cognitive load associated with manual searching or re-listening to irrelevant content [1]. User-controlled navigation enables learners to resume from precise locations, supporting selective attention and self-paced study—critical for effective cognitive engagement in long academic materials [22]. Empirical evaluations of DAISY and EPUB 3 talking-book standards also demonstrate that hierarchical navigation (by heading, page, or paragraph) increases reading speed and comprehension while lowering user frustration compared with linear playback [23]-[24].

From an engineering standpoint, integrating selective playback parallels current best practices in multimodal accessibility frameworks such as WCAG 2.2 and ISO 14289-1 (PDF/UA), which emphasize “structured content addressing” and “navigable sequential playback.” These frameworks underline that accessible readers should expose internal structure (headings, landmarks, and semantic elements) to assistive technologies, allowing playback initiation at arbitrary points without losing context [2]-[3]. Research on educational TTS interfaces corroborates that detailed playback controls—such as jump-to-paragraph, resume-at-cursor, or section bookmarks—augment user control and autonomy, which are essential for learner satisfaction and continued engagement [4]. Collectively, this body of research supports the requirement that playback be user-selectable by document segments to improve usability, accessibility, and learning efficiency.

Appendix R8: Operating Systems Support

What "operating system compatibility" means

Operating system compatibility refers to VIVID's ability to function consistently across different computing environments, regardless of the underlying OS architecture. This includes proper rendering of the web interface, smooth interaction between the browser and system resources, and consistent performance across Windows, macOS, and Linux distributions such as Ubuntu. Because VIVID is planned to be a web application, this objective ensures users can access its features without requiring OS-specific installation or configuration.

Why multi-OS support matters

Supporting multiple operating systems ensures inclusivity and accessibility for a diverse user base. Although Windows remains the dominant desktop OS, accounting for roughly 70% of global market share, macOS holds about 15%, and Linux-based systems (including Ubuntu) contribute 3–5% [25]-[26]. Collectively, these three platforms cover nearly 90% of desktop users worldwide. Moreover, as mobile and tablet devices increasingly replace traditional computers for academic and professional tasks, ensuring browser-level compatibility allows VIVID to reach users across devices running Android or iPadOS. This broad accessibility aligns with our project's goal of providing inclusive learning support across both hardware and software boundaries.

How we will measure success

Validation of this objective will involve verifying that users can reliably access and operate the VIVID web application across Windows 11, macOS Sonoma, and Ubuntu 22.04 using browsers. Testing will confirm that the interface renders correctly, that all core functionalities such as PDF upload, parsing, playback, and customization behave consistently, and that no OS-specific display or interaction issues occur.

Automated browser compatibility testing using frameworks such as Playwright or Selenium Grid simulates user sessions across these platforms to confirm consistent functionality and responsiveness. In addition, user testers on each operating system will assess visual consistency, control responsiveness, and ease of access. This objective will be considered achieved once the system can be accessed and used from the three operating systems without platform-specific defects.

Appendix R10: Responsive UI / Adaptive Layout Support

What is a responsive UI layout?
A responsive layout adjusts its structure and element sizes to fit different screen dimensions. In VIVIDpdf, this means the same codebase serves desktops and tablets without separate layouts.

Current implementation scope
The UI is designed for laptop and tablet screens. It is not optimized for phone-sized screens. During development, we determined that the primary use case — reading multi-page academic PDFs with playback controls and a settings panel — is not practical on screens with a width below ~640px. PDF pages need sufficient width to remain readable, and the player bar, settings popup, and skip zone drawing all require enough screen space for comfortable interaction.

The responsive techniques used include:

- **Fluid widths**: The player bar uses max-width: 95vw to prevent overflow on narrower screens. Modals and popups use percentage-based widths (max-width: 90%).
- **CSS Grid with auto-fill**: The recent files dashboard uses grid-template-columns: repeat(auto-fill, minmax(140px, 1fr)) to fit as many file cards as the screen allows, scaling from 2 columns on small tablets to 4+ on wide desktops.
- **Media query breakpoint at 640px**: Above this width, the file grid switches to larger cards (minmax(180px, 1fr)) with wider gaps. This is the only explicit breakpoint.
- **"*flex-wrap*" on the player bar**: Controls wrap onto a second row when horizontal space is insufficient, keeping all buttons accessible.
- **Touch event support**: Skip zone drawing and viewport panning handle onTouchStart, onTouchMove, and onTouchEnd events for tablet use with fingers or styluses.

Why not phone screens?
Reading a full PDF page on a phone screen (typically 360–414px wide) produces very small text, making it hard to click on specific words or draw skip zones. The player bar and settings pop-up together would consume most of the vertical space. In addition, people primarily use laptops or tablets for reading academic papers. Thus, phone support is listed as future work (Section 6.4).

Why responsive layouts matter?
A single responsive codebase avoids maintaining separate layouts for different devices, reducing code divergence and update effort [27]. Studies show that responsive design has a strong effect on usability — one regression analysis found it accounted for 91.5% of the variance in the tested sample [28]. User surveys also indicate that 83% of users expect applications to work consistently across their devices [29]. These findings support the decision to invest in adaptive layout even when targeting only two form factors (desktop and tablet).

Appendix R9: Non-Chromium Browsers Support

What “browser-engine compatibility” means

Browser-engine compatibility refers to VIVID’s ability to function consistently across web browsers that use different rendering engines. Each major browser interprets HTML, CSS, and JavaScript using its engine, which can cause layout or functional inconsistencies if not properly supported. The three main browser engines are Blink (used by Chrome, Edge, and Opera), WebKit (used by Safari), and Gecko (used by Firefox). Ensuring compatibility across these engines ensures that VIVID can render correctly, execute scripts reliably, and maintain an identical user experience in the most widely used browsers.

Why multi-engine support matters

Multi-engine support ensures that VIVID remains accessible across all major devices and browser ecosystems. According to StatCounter (2025), Google Chrome accounts for about 64 percent, Apple Safari for 20 percent, and Mozilla Firefox for 6 percent of global browser usage [30], together covering nearly 90 percent of users. Because Safari is the default browser on iOS and macOS, and Firefox remains popular among Linux and privacy-focused users, limiting support to Chromium-based browsers would exclude significant user groups. Supporting multiple browser engines, therefore, ensures inclusivity and long-term reliability across diverse web environments.

How we will measure success

Validation of this objective will follow a similar approach to that used for testing operating system compatibility (Appendix O3). The team will verify that users can access and use the VIVID web application from browsers powered by Blink, WebKit, and Gecko engines. Automated cross-browser testing frameworks such as Playwright, BrowserStack, or Selenium Grid may be used to evaluate rendering accuracy, interactive response, and playback consistency. In addition, selective manual testing will be performed to confirm layout alignment, TTS playback stability, and correct handling of JavaScript APIs unique to each engine. The objective will be considered achieved once the application operates reliably across the three browser engines without functionality errors.

Appendix O: UX Click-to-photon Latency

An application's responsiveness is critical for user satisfaction. Delays between user input and visual feedback can make an interface feel sluggish and, in immersive environments, even induce cybersickness [31]. Research shows that human perception of latency is task-dependent; for direct manipulation tasks like dragging, delays as short as 10-20 milliseconds can be noticeable, significantly impacting performance and user experience [31]. Therefore, our application must use optimized algorithms and efficient rendering pipelines to ensure every interaction feels immediate.

For necessary delays, such as loading data, it's crucial to manage users' perceptions of waiting. Loading animations inform the user that the system is working, which can reduce anxiety [33]. The effectiveness of a loading animation depends on the wait's duration. For short, indeterminate delays (under ~4 seconds), animated indicators like spinners are effective. They engage the user and can make the wait feel shorter than it is. For longer, determinate operations, progress bars are superior because they provide clear feedback on completion status, making the wait more tolerable by managing expectations.

Appendix T1: Element Skipping Verification

Test Objective: Verify removal of non-prose elements from text-to-speech processing via user-defined bounding boxes.

Test: The user uploads an academic journal PDF from [nh1] and uses the bottom toolbar's "Mark Skip Area" tool to draw bounding boxes over headers and footers. Audio output is evaluated to verify the marked regions are excluded from speech.

Results (PASS): Figure 12 displays the manual marking interface, using red dashed regions to define user-specified skip areas targeting the top header, right vertical margin text, and bottom footer. Audio playback transcripts confirm the complete exclusion of text within these drawn zones.

Figure 12. Screenshot of sample PDF page with skipped areas.

Appendix T2: Context-Aware Semantics Verification

Test Objective: Verify the system's ability to translate mathematical notation into natural, context-appropriate speech.

Test: An MAT187 calculus textbook PDF containing complex power functions and derivatives was processed using "AI Fix Mode." The output was monitored via the developer console to compare the raw text against the generated "TextToSpeak" string.

Results (PASS): Figure 14 shows the user interface used to enable the semantic processing mode. Figure 13 displays the debug console, where the mathematical expression $\frac{d}{dx}(f(x)g(x)) = \frac{d}{dx}(x^7) = 7x^6$ is highlighted. The transcript confirms the system correctly articulated the notation as "*the derivative with respect to x of f of x times g of x equals the derivative with respect to x of x to the seventh power equals seven x to the sixth power*" rather than reading literal characters.

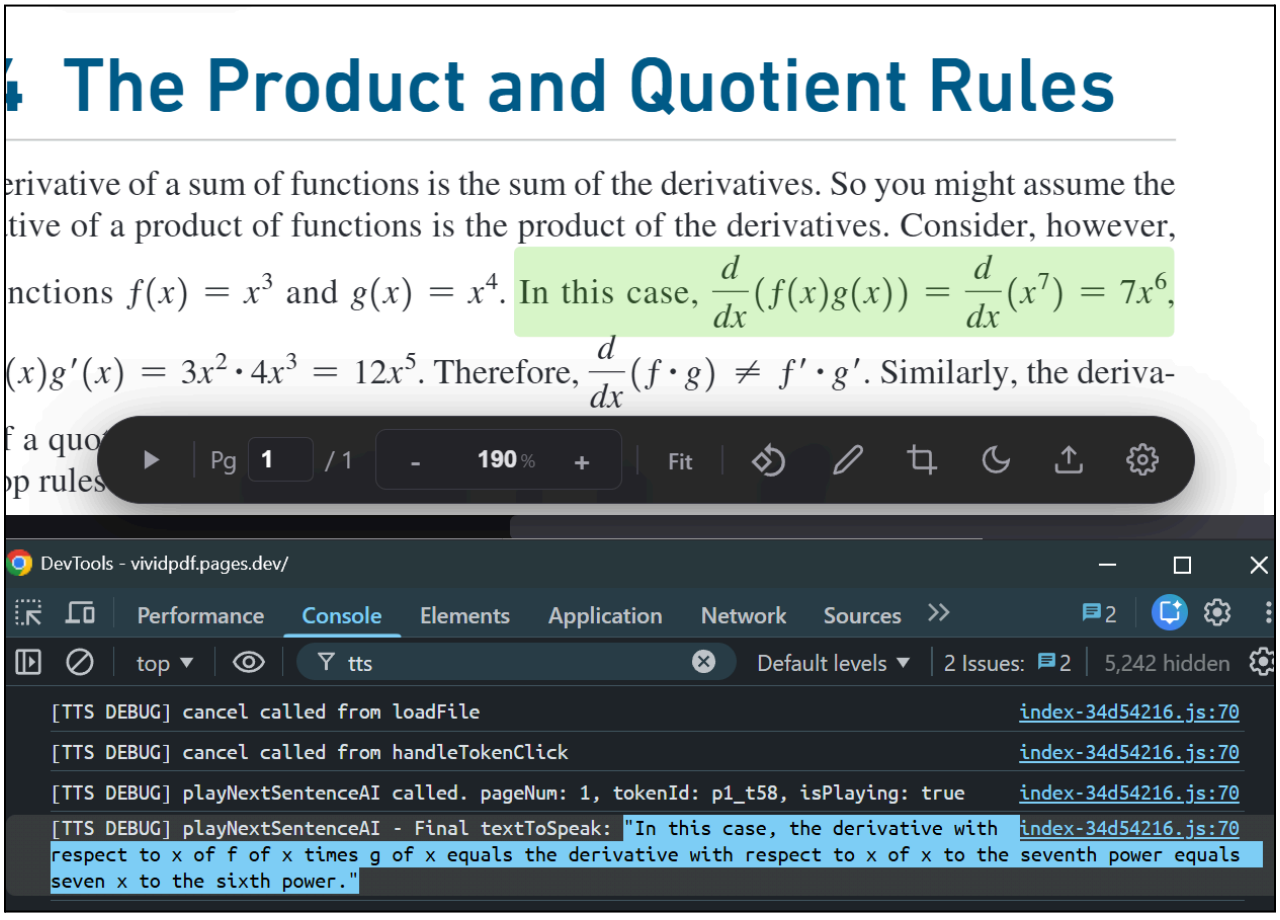


Figure 13. Generated transcript in debug console, highlighted in blue.

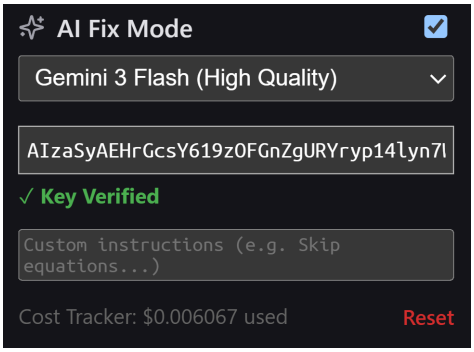


Figure 14. Screenshot of UI for configuring AI Fix Mode, which allows reading equations.

Appendix T3: User-Defined Pronunciation Verification

Test Objective: Verify that custom pronunciation rules correctly override default text-to-speech output.

Test: Normally, the TTS wrongly pronounces “pmos” as “P-M-O-S.” A custom rule was created to replace the word "pmos" with the pronunciation "P moss." The system's internal speech buffer was inspected to confirm the replacement occurred before audio generation.

Results (PASS): Figure 15 shows the configuration menu where the substitution rule was defined. Figure 16 displays the developer console transcript, which confirms the "ACTUAL SPOKEN SENTENCE" replaced all instances of "pmos" with "P moss." The rule remained active and functional across multiple page loads.

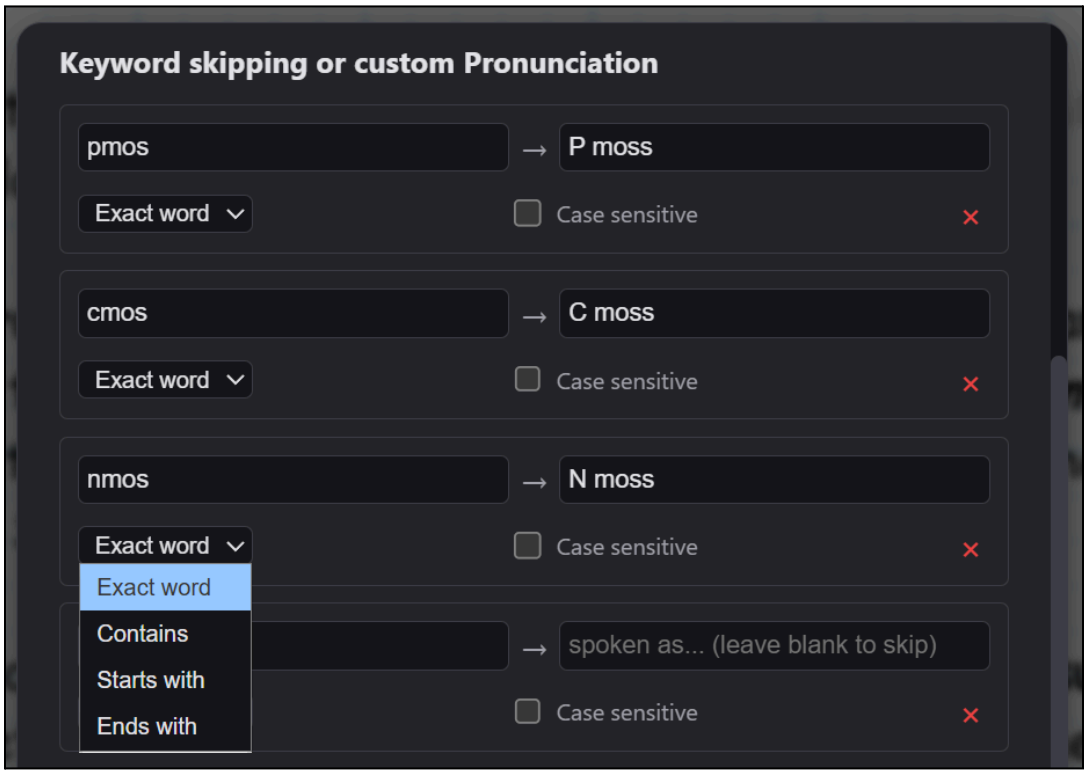


Figure 15. Keyword skipping and custom pronunciation settings menu showing the "pmos → P moss" rule.

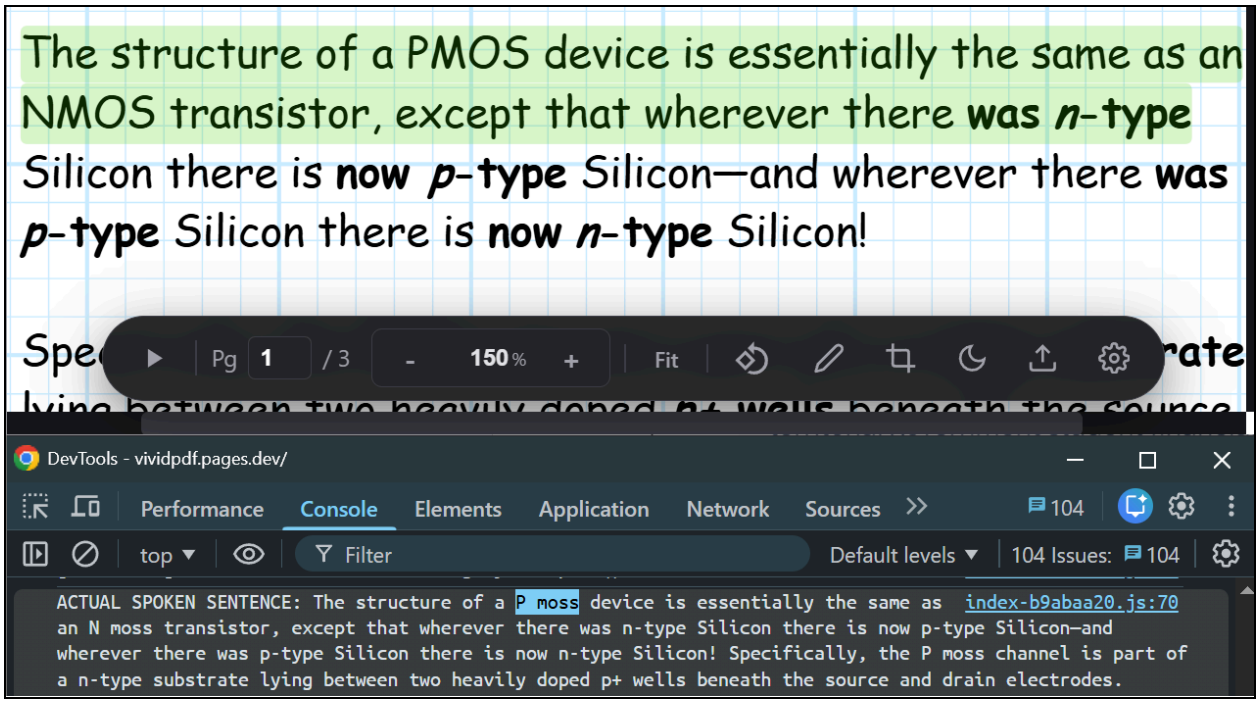


Figure 16: Debug console output showing the final spoken transcript with active keyword substitutions highlighted.

Appendix T4: Custom Skipping Rules Verification

Test Objective: Verify the system successfully ignores user-defined text patterns during audio playback.

Test: Custom skipping rules were configured via the UI menu to exclude specific text patterns, including URLs, email addresses, content inside parentheses, and various citation formats. Multiple academic PDFs containing these elements were then processed to evaluate the audio output.

Results (PASS): Audio playback confirmed that all user-defined patterns were successfully excluded from speech. Figure 17 displays the GUI configuration menu used to define the rules. Figures 18 through 20 demonstrate the system correctly identifying and skipping the targeted text elements (superscript, IEEE, and Chicago styles) without omitting standard prose.

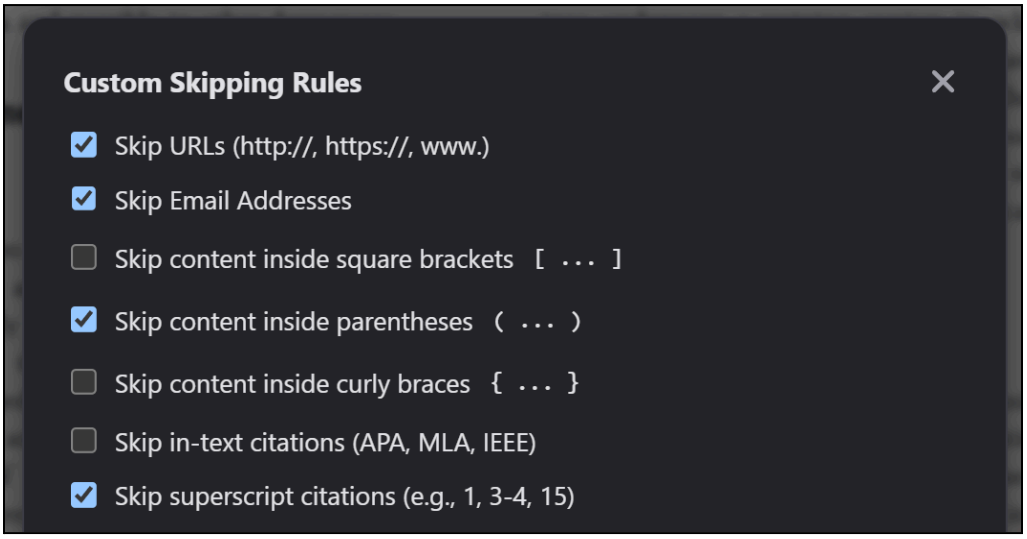
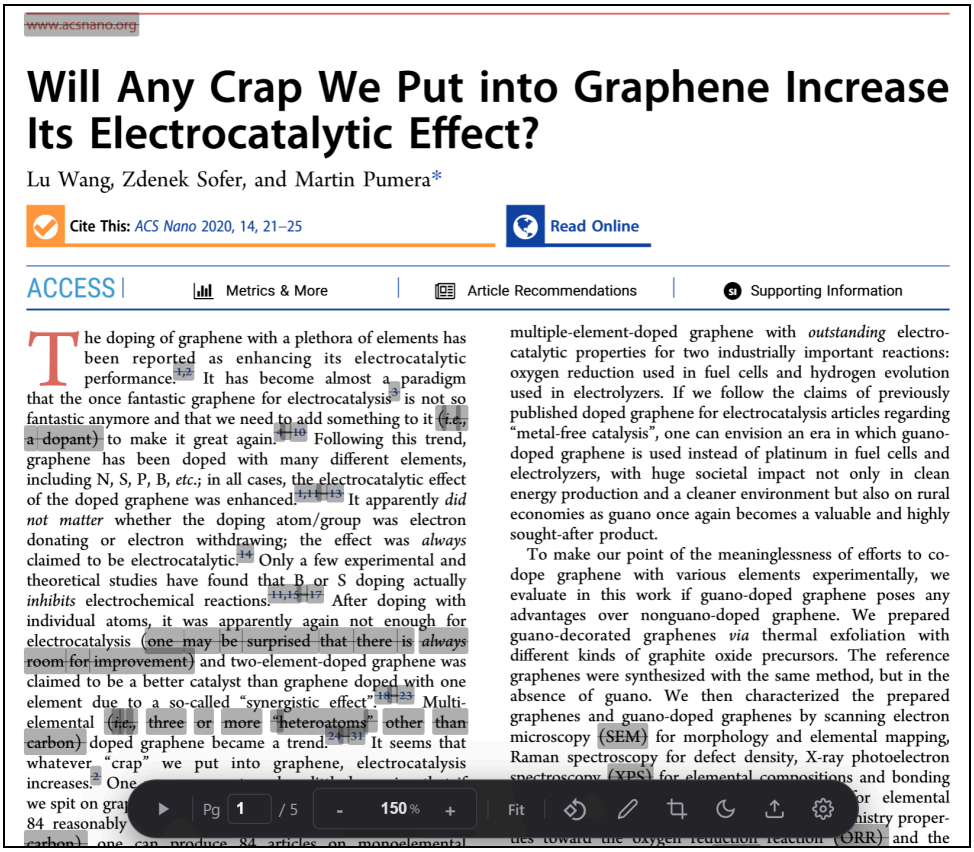


Figure 17. Custom Skipping Rules configuration menu showing selected options for skipping URLs, email addresses, parentheses, and superscript citations.



www.acsnano.org

Will Any Crap We Put into Graphene Increase Its Electrocatalytic Effect?

Lu Wang, Zdenek Sofer, and Martin Pumera*

Cite This: ACS Nano 2020, 14, 21−25 Read Online

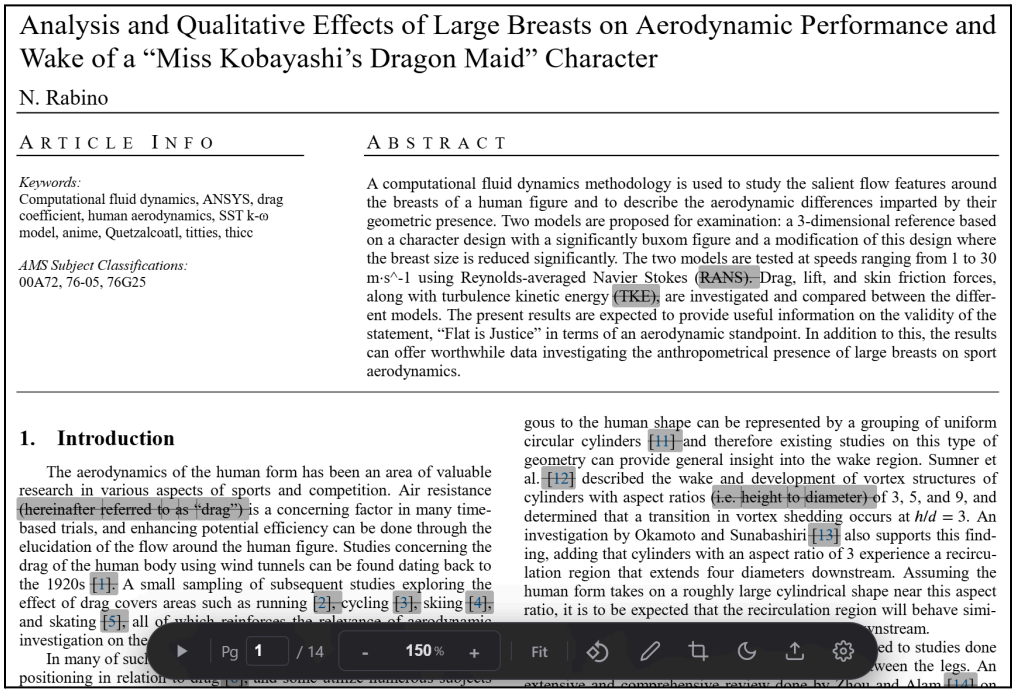
ACCESS | Metrics & More | Article Recommendations | Supporting Information

The doping of graphene with a plethora of elements has been reported as enhancing its electrocatalytic performance.[1,2] It has become almost a paradigm that the once fantastic graphene for electrocatalysis[3] is not so fantastic anymore and that we need to add something to it (i.e., a dopant) to make it great again.[4−10] Following this trend, graphene has been doped with many different elements, including N, S, P, B, *etc.*; in all cases, the electrocatalytic effect of the doped graphene was enhanced.[7,11−13] It apparently *did not matter* whether the doping atom/group was electron donating or electron withdrawing; the effect was *always* claimed to be electrocatalytic.[14] Only a few experimental and theoretical studies have found that B or S doping actually *inhibits* electrochemical reactions.[11,15−17] After doping with individual atoms, it was apparently again not enough for electrocatalysis (one may be surprised that there is *always* room for improvement) and two-element-doped graphene was claimed to be a better catalyst than graphene doped with one element due to a so-called "synergistic effect".[18−23] Multielemental (i.e., three or more "heteroatoms" other than carbon) doped graphene became a trend.[24−31] It seems that whatever "crap" we put into graphene, electrocatalysis increases.[2] One
we spit on gra
84 reasonably
carbon), one can produce 84 articles on monoelemental

multiple-element-doped graphene with *outstanding* electrocatalytic properties for two industrially important reactions: oxygen reduction used in fuel cells and hydrogen evolution used in electrolyzers. If we follow the claims of previously published doped graphene for electrocatalysis articles regarding "metal-free catalysis", one can envision an era in which guano-doped graphene is used instead of platinum in fuel cells and electrolyzers, with huge societal impact not only in clean energy production and a cleaner environment but also on rural economies as guano once again becomes a valuable and highly sought-after product.

To make our point of the meaninglessness of efforts to co-dope graphene with various elements experimentally, we evaluate in this work if guano-doped graphene poses any advantages over nonguano-doped graphene. We prepared guano-decorated graphenes *via* thermal exfoliation with different kinds of graphite oxide precursors. The reference graphenes were synthesized with the same method, but in the absence of guano. We then characterized the prepared graphenes and guano-doped graphenes by scanning electron microscopy (SEM) for morphology and elemental mapping, Raman spectroscopy for defect density, X-ray photoelectron spectroscopy (XPS) for elemental compositions and bonding
for elemental
istry proper-
ties toward the oxygen reduction reaction (ORR) and the

Figure 18: Screenshot demonstrating the successful skipping of parenthetical text, superscript in-text citations, and URLs.



Analysis and Qualitative Effects of Large Breasts on Aerodynamic Performance and Wake of a "Miss Kobayashi's Dragon Maid" Character

N. Rabino

ARTICLE INFO

Keywords:
Computational fluid dynamics, ANSYS, drag coefficient, human aerodynamics, SST k-ω model, anime, Quetzalcoatl, titties, thicc

AMS Subject Classifications:
00A72, 76-05, 76G25

ABSTRACT

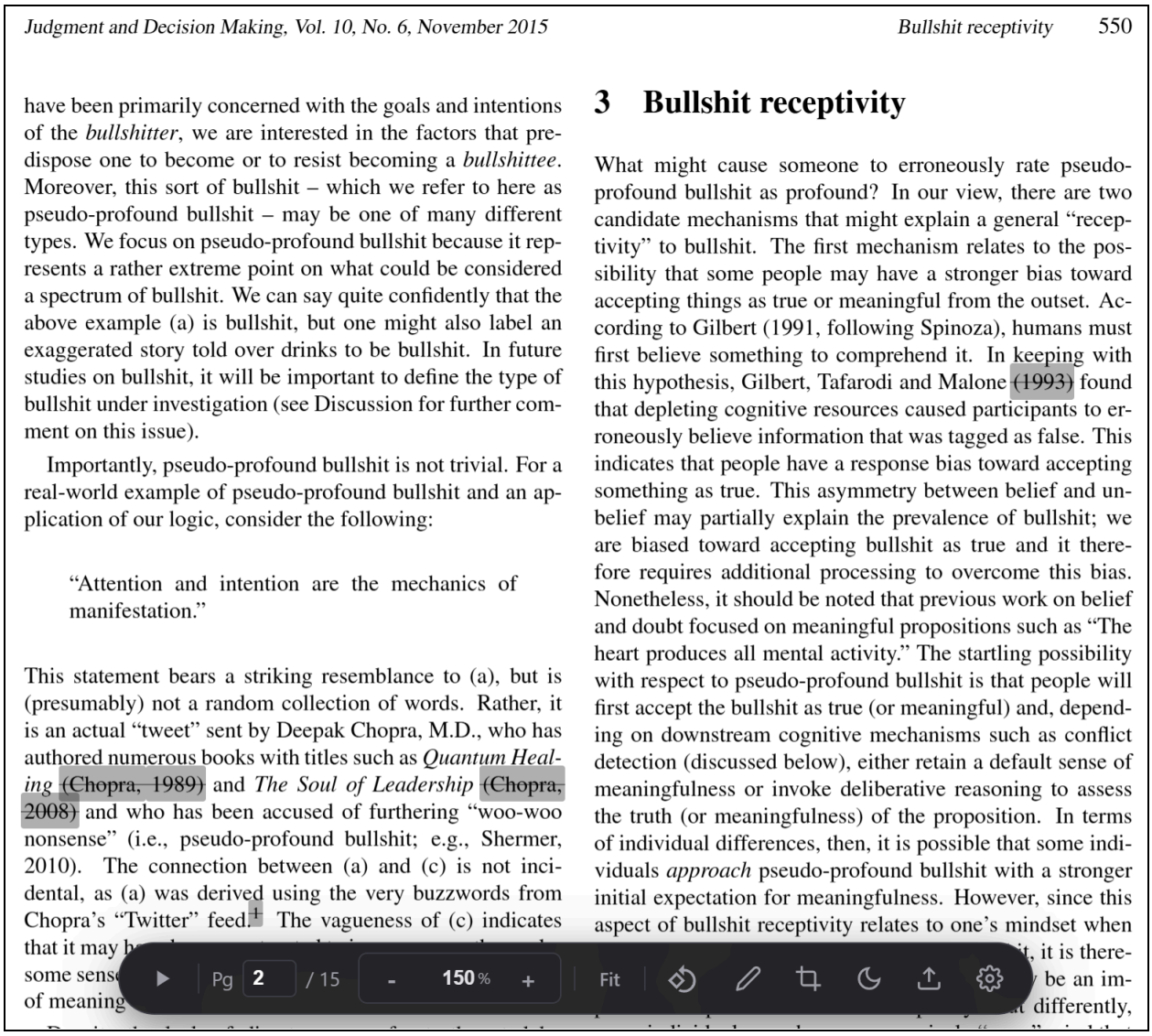
A computational fluid dynamics methodology is used to study the salient flow features around the breasts of a human figure and to describe the aerodynamic differences imparted by their geometric presence. Two models are proposed for examination: a 3-dimensional reference based on a character design with a significantly buxom figure and a modification of this design where the breast size is reduced significantly. The two models are tested at speeds ranging from 1 to 30 m·s^-1 using Reynolds-averaged Navier Stokes (RANS). Drag, lift, and skin friction forces, along with turbulence kinetic energy (TKE), are investigated and compared between the different models. The present results are expected to provide useful information on the validity of the statement, "Flat is Justice" in terms of an aerodynamic standpoint. In addition to this, the results can offer worthwhile data investigating the anthropometrical presence of large breasts on sport aerodynamics.

1. Introduction

The aerodynamics of the human form has been an area of valuable research in various aspects of sports and competition. Air resistance (hereinafter referred to as "drag") is a concerning factor in many time-based trials, and enhancing potential efficiency can be done through the elucidation of the flow around the human figure. Studies concerning the drag of the human body using wind tunnels can be found dating back to the 1920s [1]. A small sampling of subsequent studies exploring the effect of drag covers areas such as running [2], cycling [3], skiing [4], and skating [5], all of which reinforces the relevance of aerodynamic investigation on the
In many of such
positioning in relation to drag [6], and some utilize numerous subjects

gous to the human shape can be represented by a grouping of uniform circular cylinders [11] and therefore existing studies on this type of geometry can provide general insight into the wake region. Sumner et al. [12] described the wake and development of vortex structures of cylinders with aspect ratios (i.e. height to diameter) of 3, 5, and 9, and determined that a transition in vortex shedding occurs at $h/d = 3$. An investigation by Okamoto and Sunabashiri [13] also supports this finding, adding that cylinders with an aspect ratio of 3 experience a recirculation region that extends four diameters downstream. Assuming the human form takes on a roughly large cylindrical shape near this aspect ratio, it is to be expected that the recirculation region will behave simi-
ynstream.
ed to studies done
ween the legs. An
extensive and comprehensive review done by Zhou and Alam [14] on

Figure 19: Screenshot demonstrating the skipping of IEEE bracketed citations.



have been primarily concerned with the goals and intentions of the *bullshitter*, we are interested in the factors that predispose one to become or to resist becoming a *bullshittee*. Moreover, this sort of bullshit – which we refer to here as pseudo-profound bullshit – may be one of many different types. We focus on pseudo-profound bullshit because it represents a rather extreme point on what could be considered a spectrum of bullshit. We can say quite confidently that the above example (a) is bullshit, but one might also label an exaggerated story told over drinks to be bullshit. In future studies on bullshit, it will be important to define the type of bullshit under investigation (see Discussion for further comment on this issue).

Importantly, pseudo-profound bullshit is not trivial. For a real-world example of pseudo-profound bullshit and an application of our logic, consider the following:

> "Attention and intention are the mechanics of manifestation."

This statement bears a striking resemblance to (a), but is (presumably) not a random collection of words. Rather, it is an actual "tweet" sent by Deepak Chopra, M.D., who has authored numerous books with titles such as *Quantum Healing* (Chopra, 1989) and *The Soul of Leadership* (Chopra, 2008) and who has been accused of furthering "woo-woo nonsense" (i.e., pseudo-profound bullshit; e.g., Shermer, 2010). The connection between (a) and (c) is not incidental, as (a) was derived using the very buzzwords from Chopra's "Twitter" feed.[1] The vagueness of (c) indicates that it may h
some sens
of meaning

3 Bullshit receptivity

What might cause someone to erroneously rate pseudo-profound bullshit as profound? In our view, there are two candidate mechanisms that might explain a general "receptivity" to bullshit. The first mechanism relates to the possibility that some people may have a stronger bias toward accepting things as true or meaningful from the outset. According to Gilbert (1991, following Spinoza), humans must first believe something to comprehend it. In keeping with this hypothesis, Gilbert, Tafarodi and Malone (1993) found that depleting cognitive resources caused participants to erroneously believe information that was tagged as false. This indicates that people have a response bias toward accepting something as true. This asymmetry between belief and unbelief may partially explain the prevalence of bullshit; we are biased toward accepting bullshit as true and it therefore requires additional processing to overcome this bias. Nonetheless, it should be noted that previous work on belief and doubt focused on meaningful propositions such as "The heart produces all mental activity." The startling possibility with respect to pseudo-profound bullshit is that people will first accept the bullshit as true (or meaningful) and, depending on downstream cognitive mechanisms such as conflict detection (discussed below), either retain a default sense of meaningfulness or invoke deliberative reasoning to assess the truth (or meaningfulness) of the proposition. In terms of individual differences, then, it is possible that some individuals *approach* pseudo-profound bullshit with a stronger initial expectation for meaningfulness. However, since this aspect of bullshit receptivity relates to one's mindset when

Figure 20: Screenshot demonstrating the targeted skipping of Chicago author-date citations without excluding all parenthetical content.

Appendix T5: Selective Playback Verification Test

Objective: Verify the system allows users to start audio playback from a manually selected word or sentence within the document.

Test: A user clicks a specific word mid-paragraph in a PDF to initiate playback. The developer console is monitored to confirm the text-to-speech engine immediately processes the text starting exactly from the selected word.

Results (PASS): Audio playback began exactly at the user-selected location. Figure 21 displays the document interface with the targeted word ("bombing") highlighted. The debug console transcript confirms the audio engine received and played the text starting from that precise point, ignoring all prior text on the page.

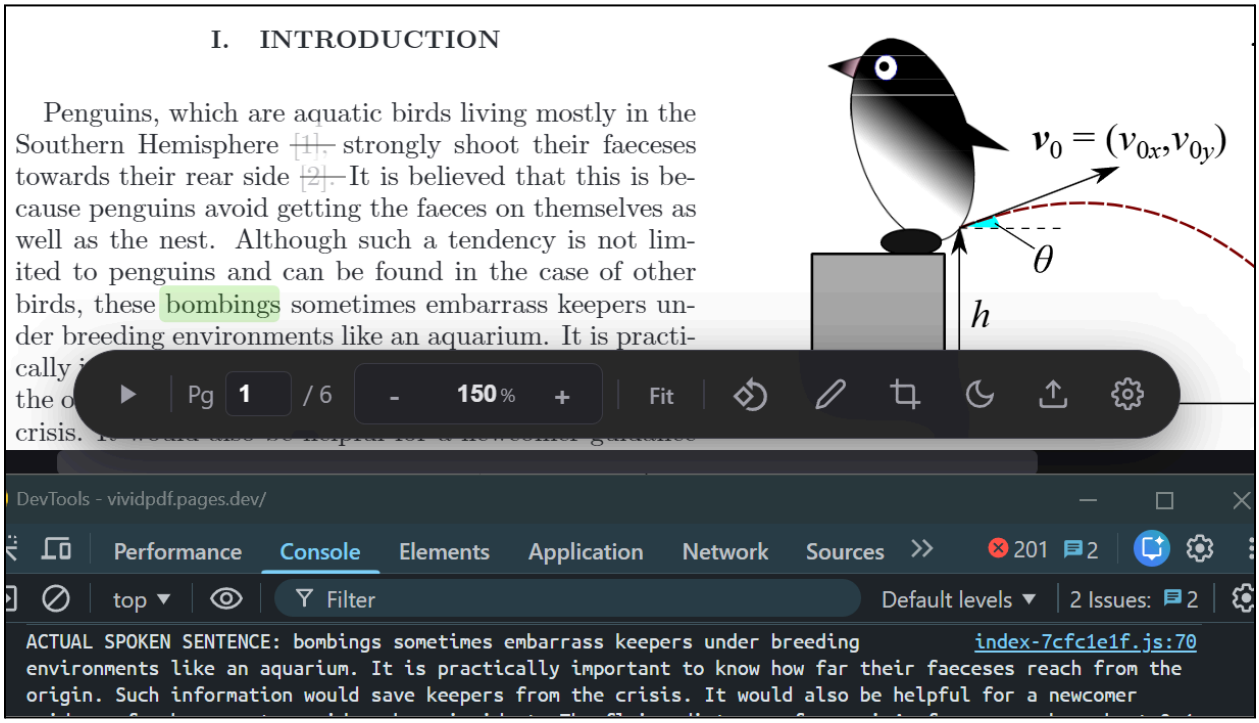


Figure 21: Screenshot of the document interface showing the specific word "bombings" clicked to initiate playback, and the debug console confirming the spoken sentence starts exactly at that word.

Appendix T6: Web Performance and Accessibility Verification

Test Objective: Verify the application meets high industry standards for loading performance and accessibility using automated auditing tools.

Test: The production application URL was evaluated using Google PageSpeed Insights, configured for desktop analysis. Overall scores and detailed metrics for both performance and accessibility were recorded.

Results: The system passed the audits with near-perfect scores. Figure 22 shows a Performance score of 99, with excellent core metrics including a First Contentful Paint (FCP) of 0.6 seconds and 0ms Total Blocking Time. Figure 23 confirms an Accessibility score of 100, showing that all automated accessibility checks (such as contrast ratios, ARIA attributes, and discernible link names) successfully passed.

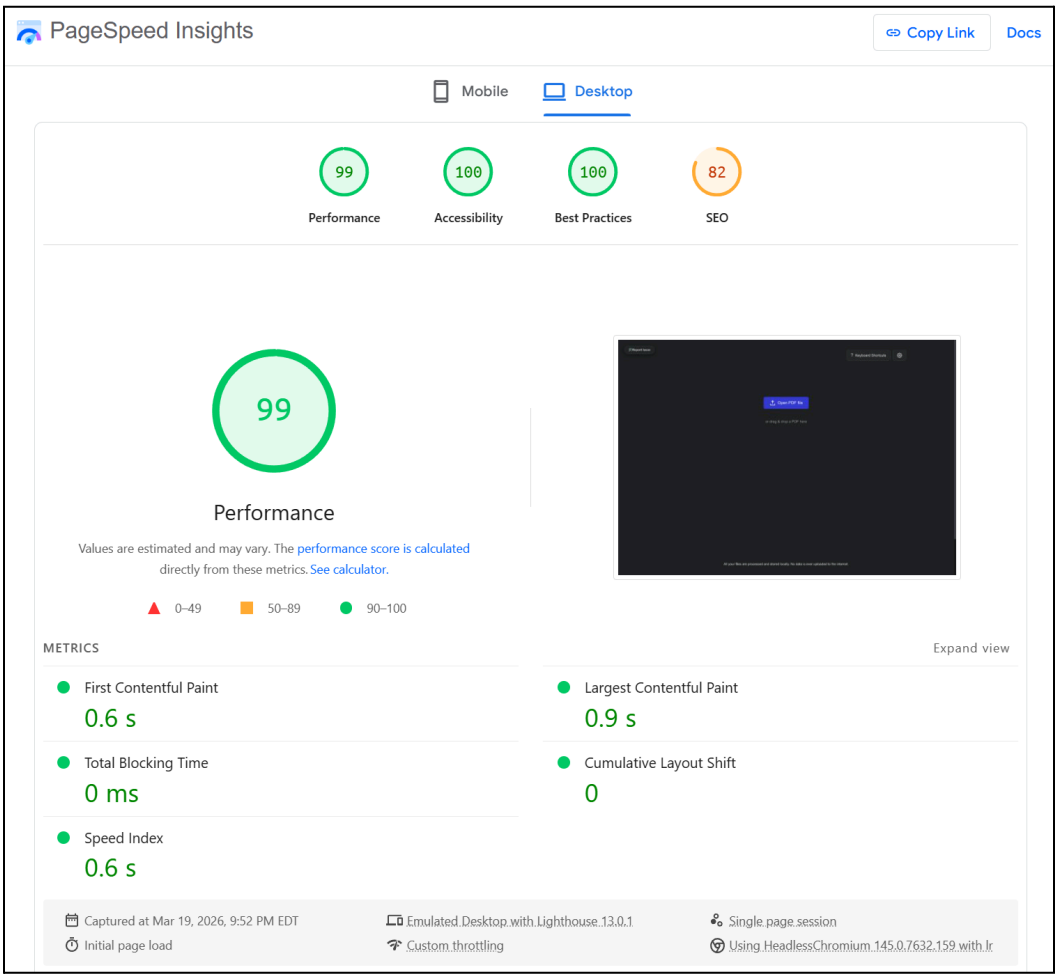


Figure 22. Google PageSpeed Insights desktop performance report showing an overall score of 99 and fast load metrics.

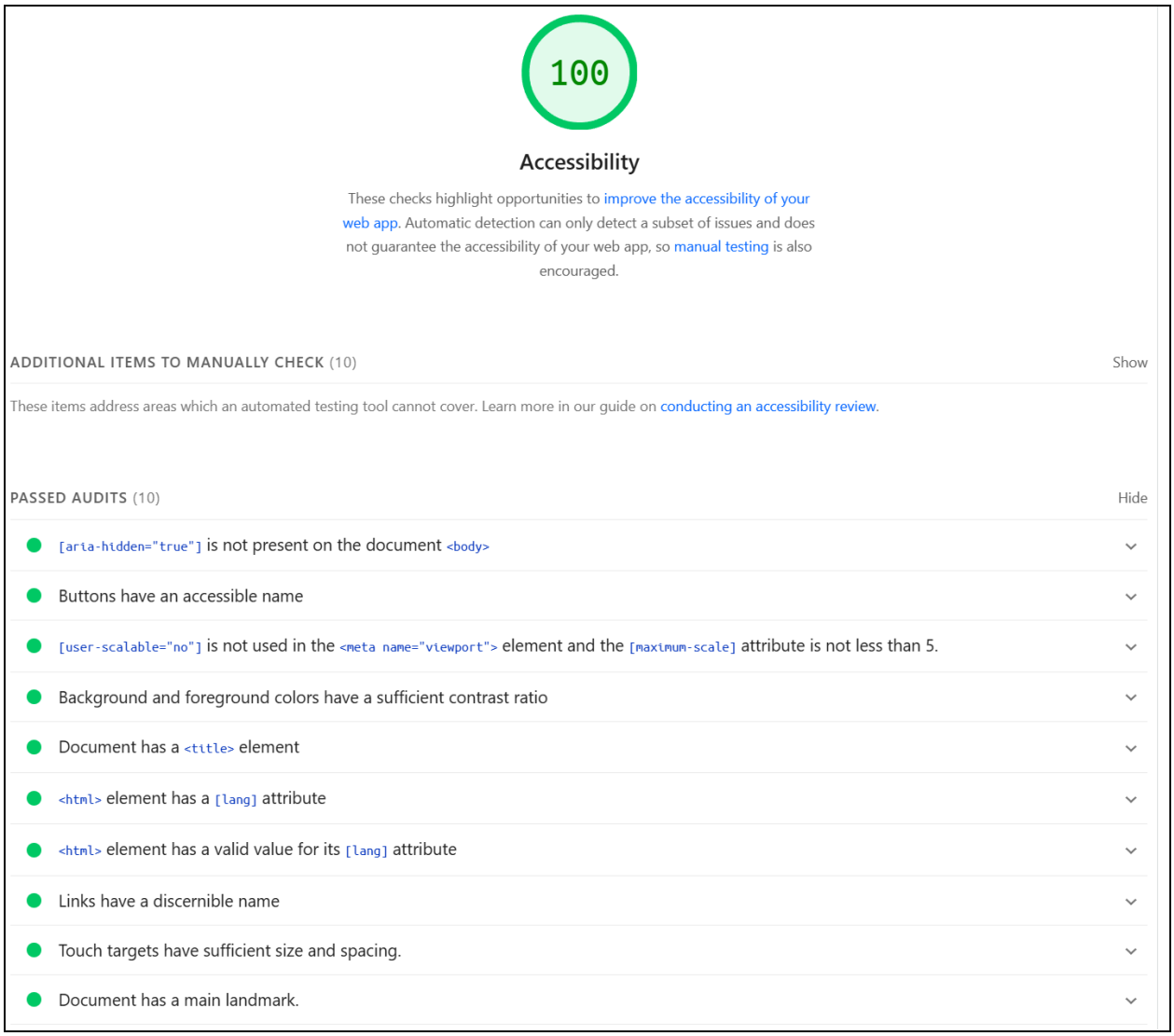


Figure 23: Google PageSpeed Insights accessibility report showing a score of 100 and the list of passed automated audits.

Appendix T7: Playback Settings Verification

Test Objective: Verify the system supports user selection of voice and real-time adjustment of playback speed during audio playback.

Test: A sample PDF was loaded and playback was started. During playback, the selected voice was changed through the playback settings menu, and the playback speed was adjusted to multiple values. The system was observed to confirm that audio output updated correctly without crashes, playback interruption, or UI malfunction.

Results (PASS): The system successfully supported both voice selection and real-time playback speed adjustment. Figure 24 shows the playback settings interface with the voice and speed controls. Figure 25 shows playback continuing normally after both settings were changed, confirming that the updated voice selection and playback speed were applied successfully.

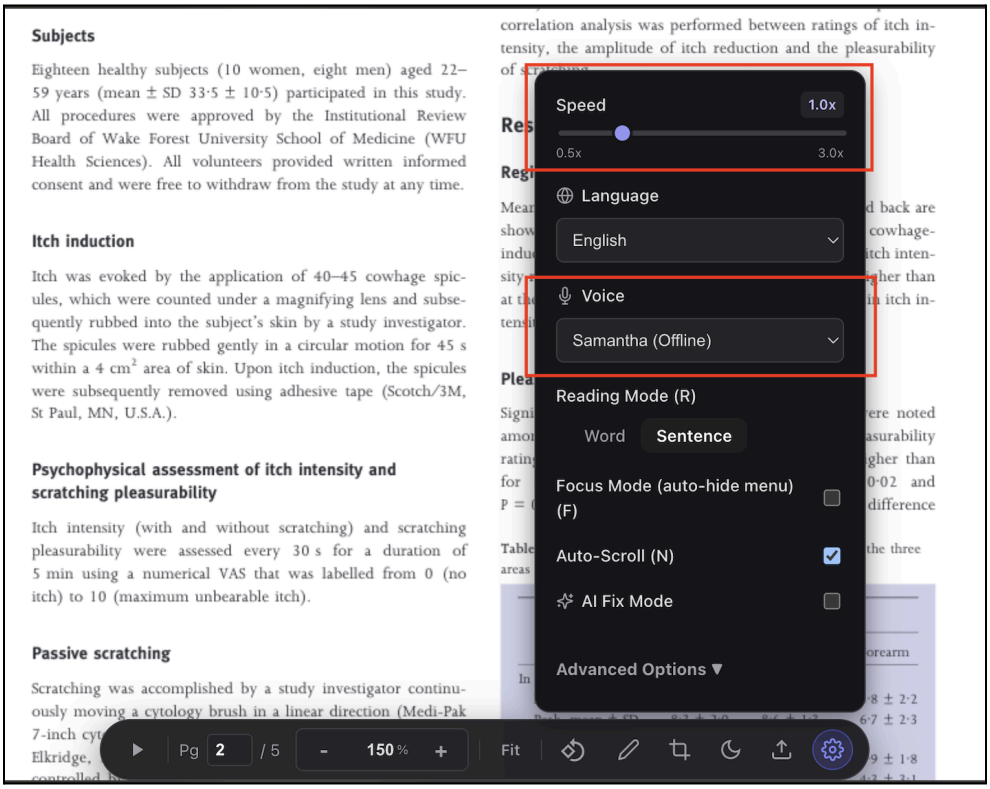


Figure 24. The playback settings menu shows the voice selection options and playback speed control.

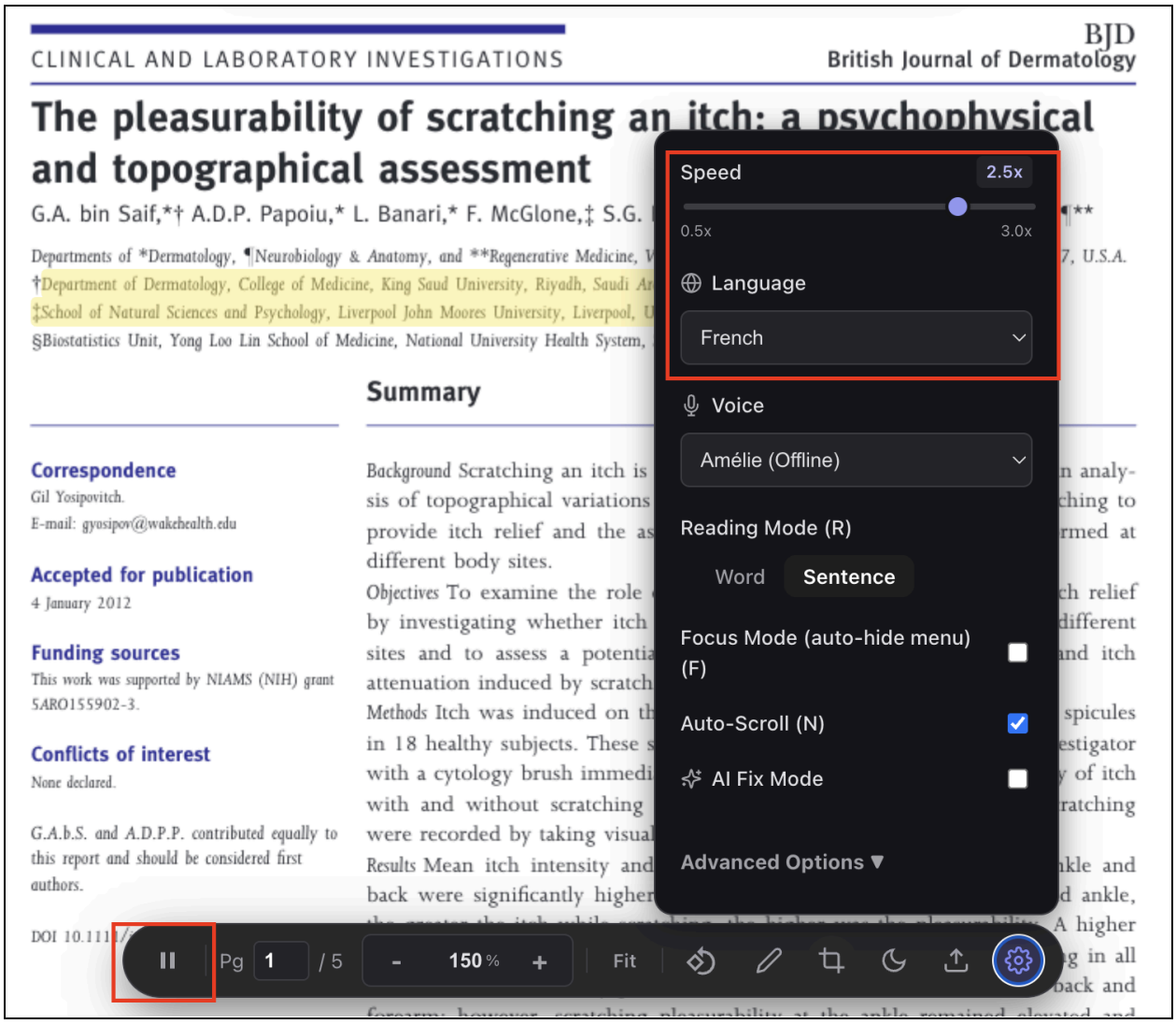


Figure 25. Playback continues normally after changing the selected voice and playback speed.

Appendix T8: Multi-OS Support Verification

Test Objective: Verify that the system functions correctly across at least three operating systems.

Test: The application was run on three operating systems(macOS, iPadOS, Windows) and tested using the same PDF workflow. For each platform, document upload, PDF rendering, text parsing, and audio playback were exercised and compared for behavioural consistency.

Results (PASS): The system functioned correctly on all tested operating systems. Figures 26 through 28 show the successful execution of the application on three different OS platforms, with consistent behaviour in document upload, rendering, parsing, and playback.

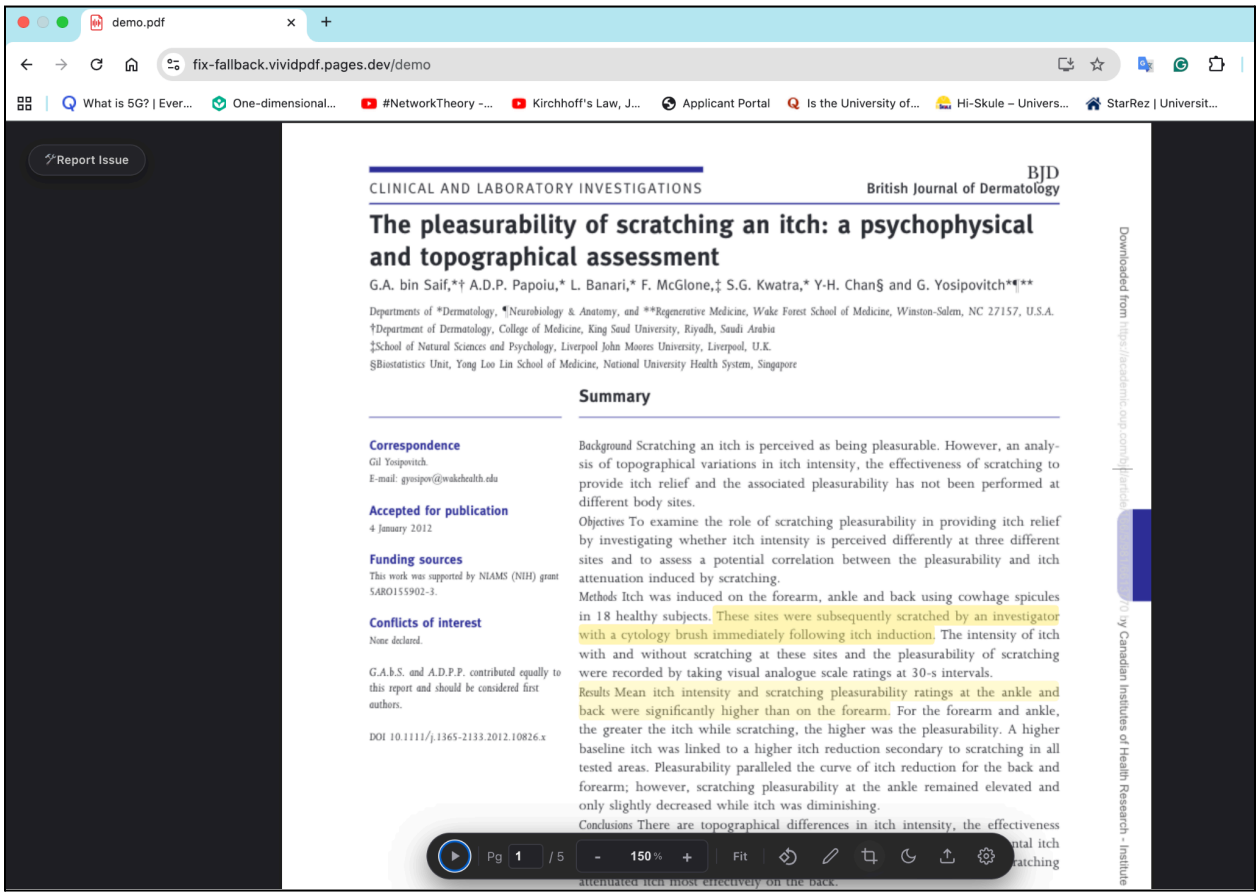


Figure 26. Application running on macOS 15.7.4, showing successful PDF rendering and active playback with real-time highlighting.

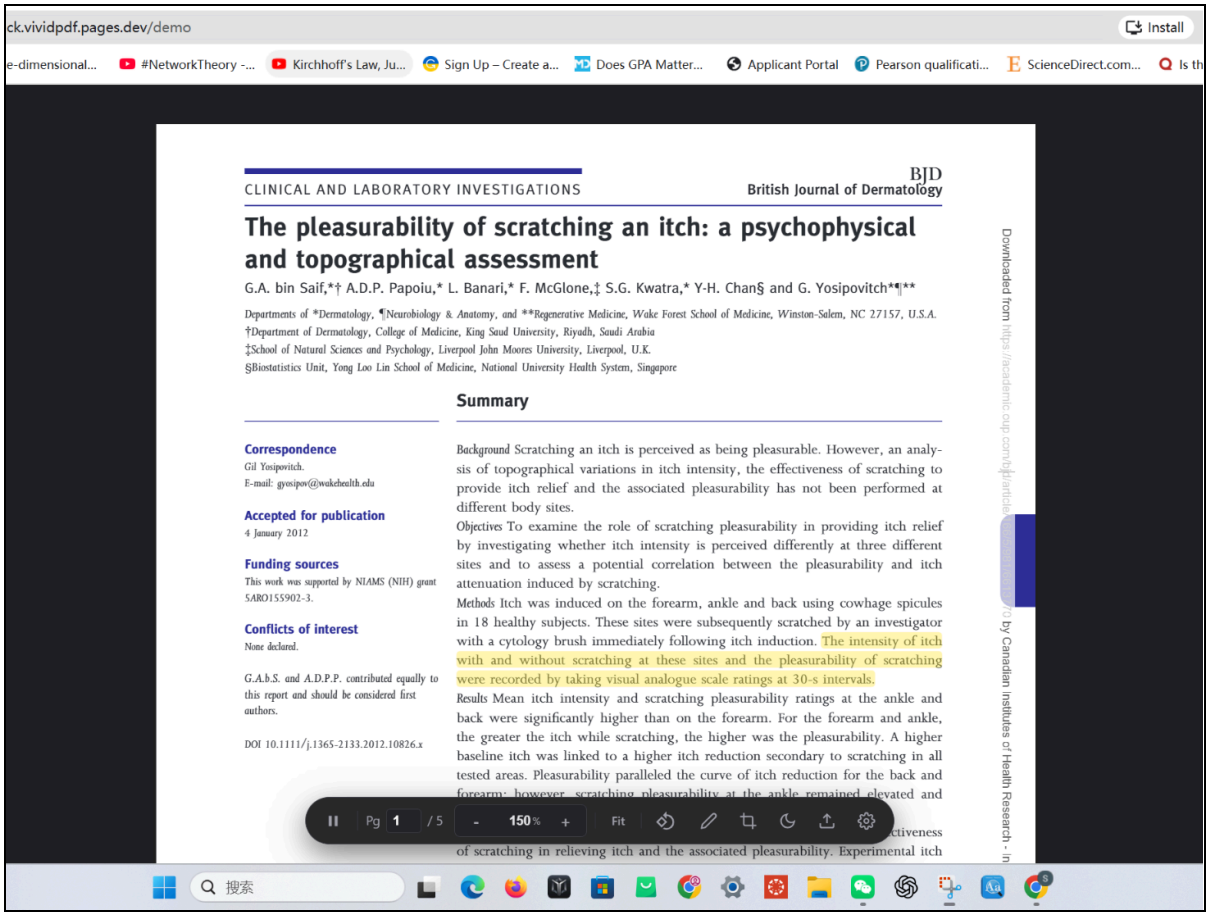


Figure 27. Application running on Windows 11, demonstrating consistent document rendering and playback behaviour.

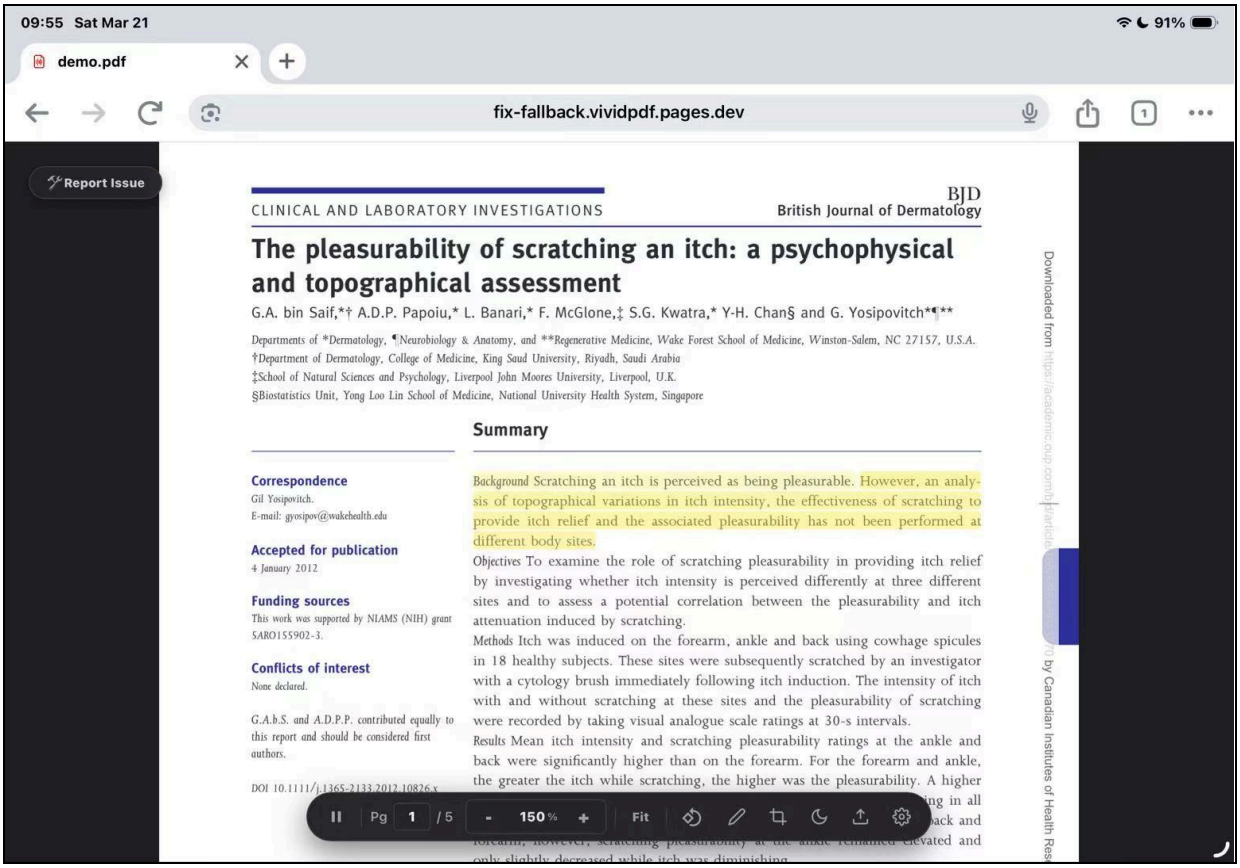


Figure 28. Application running on iPadOS 26.3.1, confirming consistent parsing and playback functionality.

Appendix T9: Multi-Browser Engine Support Verification

Test Objective: Verify that the system functions correctly across at least three browser engines.

Test: The application was tested in Chrome, Safari, and Firefox using the same PDF input and interaction flow. For each browser, document rendering, parsing, interactive highlighting, and audio playback were checked for correct operation.

Results (PASS): The system operated correctly across all tested browsers. Figures 29 through 31 show successful execution in Chrome, Safari, and Firefox, with consistent rendering, functionality, and playback behaviour.

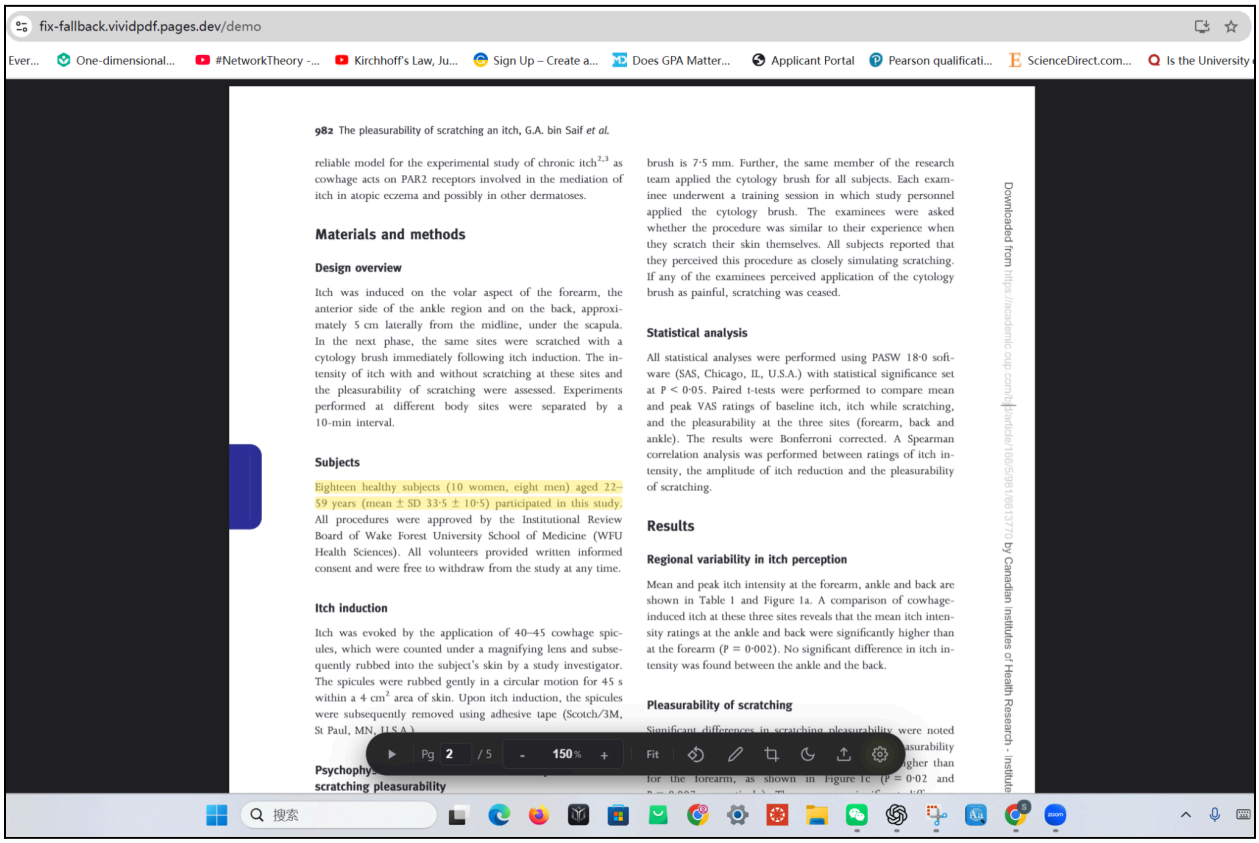


Figure 29. Screenshot of the application running successfully in Chrome.

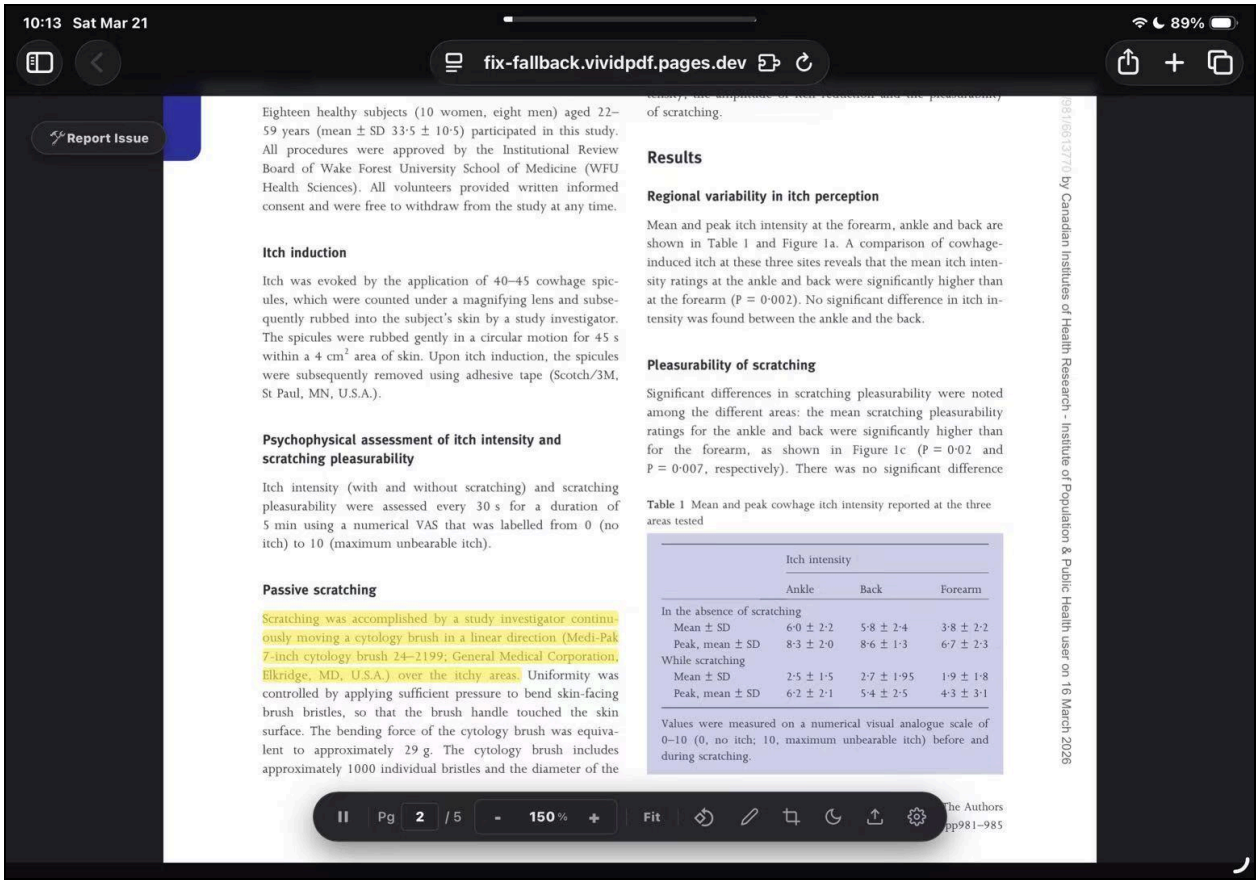


Figure 30. Screenshot of the application running successfully in Safari.

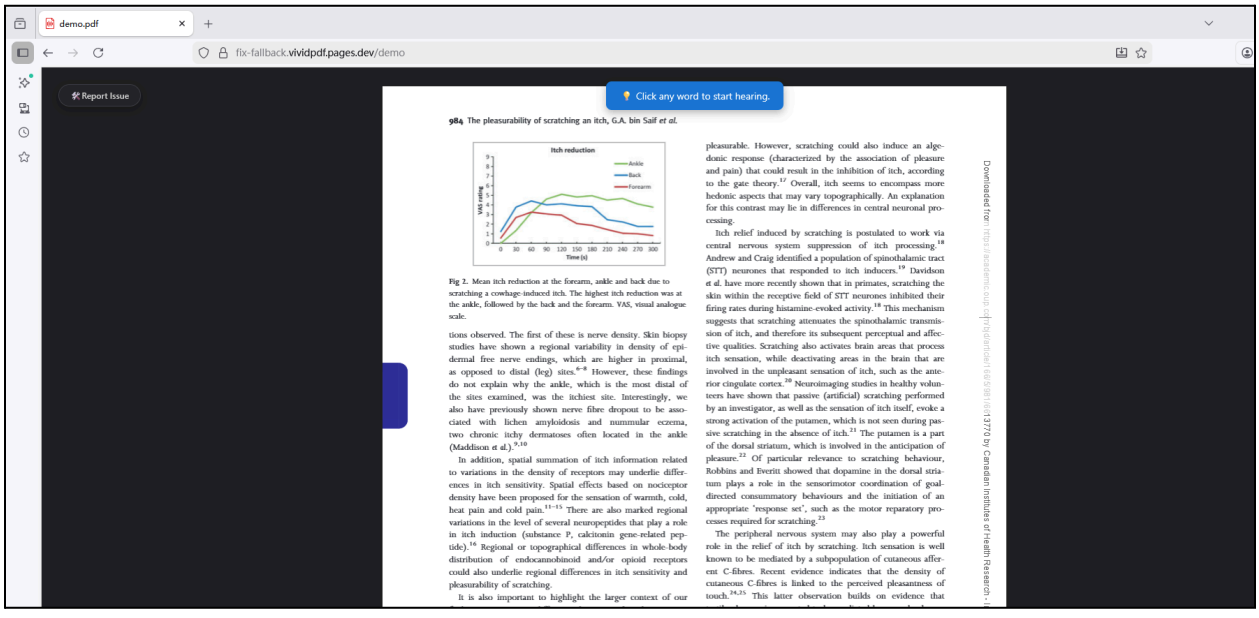


Figure 31. Screenshot of the application running successfully in Firefox.

Appendix T11: Memory Integrity Verification

Test Objective: Verify the application correctly manages browser local storage and releases memory after file deletion, preventing storage leaks.

Test: Monitored the web application's storage usage via Chrome Site Settings during three states: initial load, after populating the application with multiple PDF files to near capacity, and after executing the "Clear All PDF Contents" command within the app.

Results: The system released allocated storage as expected. Figure 32 displays the initial baseline storage of 6.6 MB. Figure 33 shows storage usage peaking at 1.9 GB after file ingestion. Figure 34 confirms usage dropped to 6.9 MB after file deletion, indicating no persistent memory leaks.

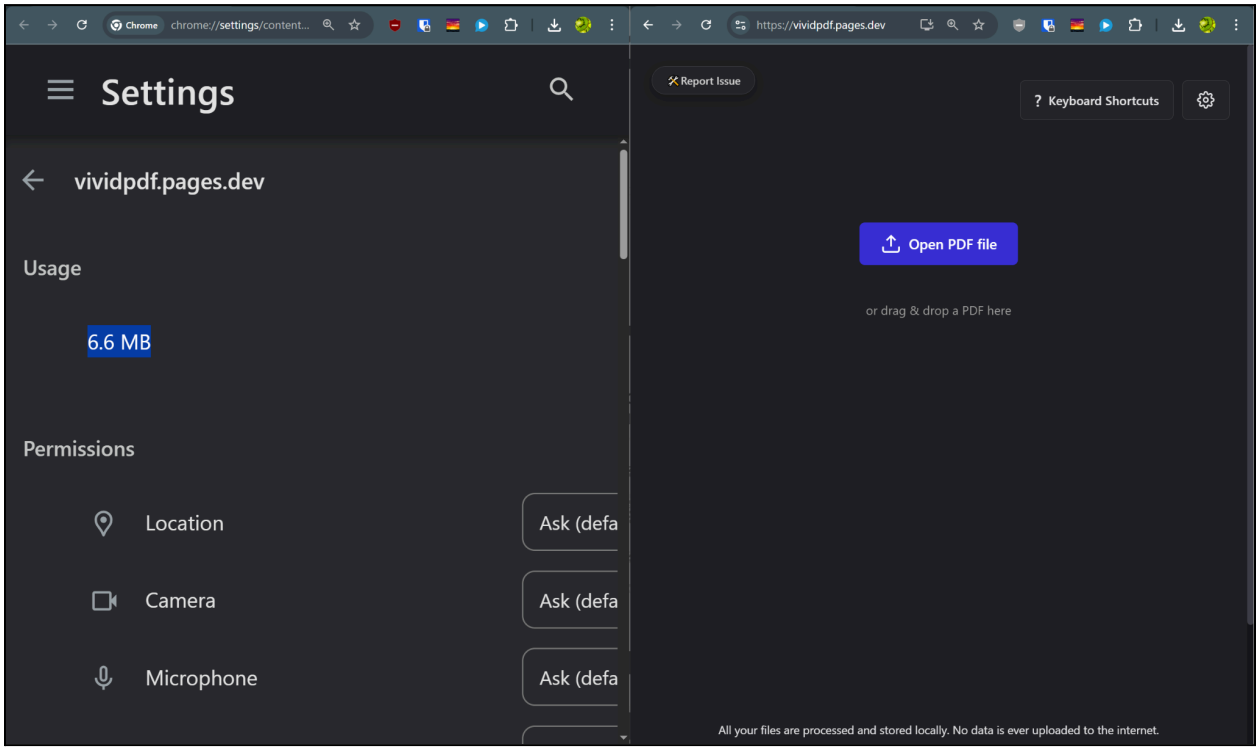


Figure 32. Chrome settings and application interface showing initial baseline storage usage of 6.6 MB.

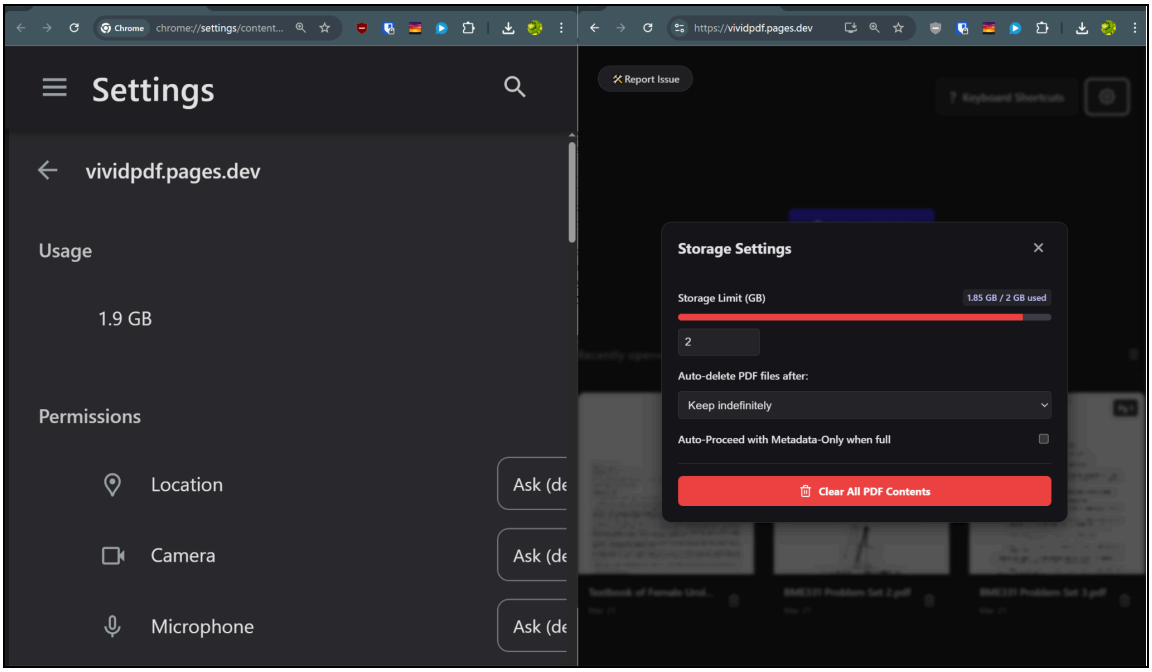


Figure 33. Chrome settings and application interface showing storage usage at 1.9 GB after loading multiple files.

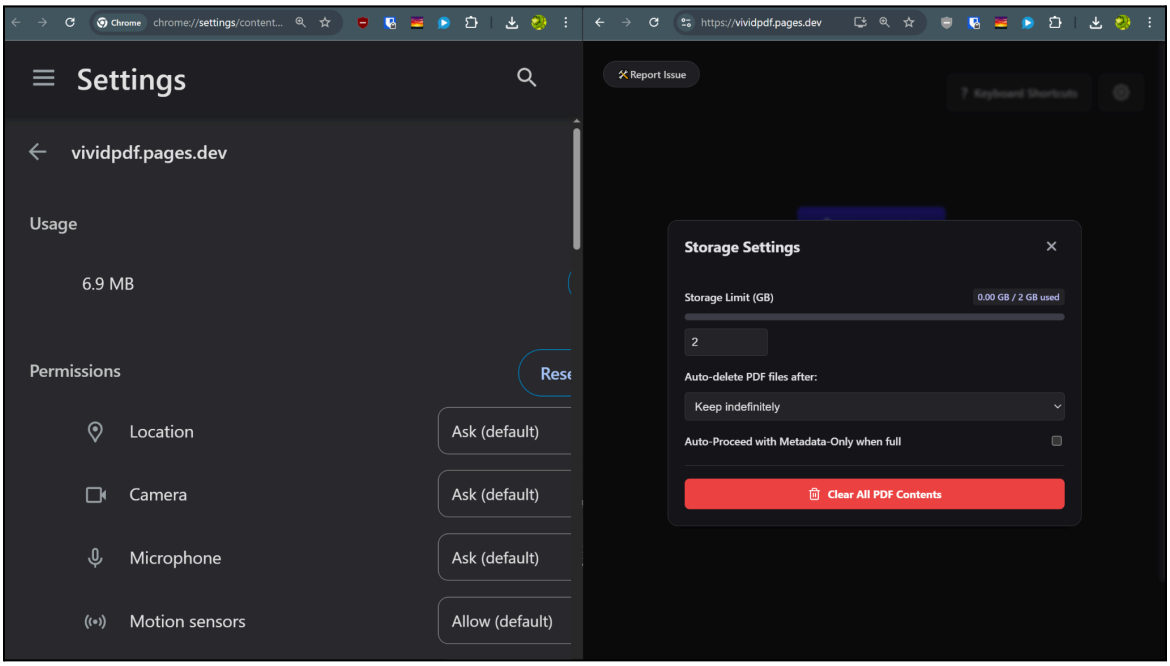


Figure 34. Chrome settings and application interface showing storage usage returned to 6.9 MB following file deletion.

Appendix T12: Multi-Language Accessibility Verification

Test Objective: Verify the application successfully processes and vocalizes text in at least three distinct languages.

Test: A PDF document containing sentences written in English, French, and Chinese was uploaded. The application's language selection tool was utilized to configure the text-to-speech engine for each corresponding language block during playback.

Results: Audio playback confirmed accurate pronunciation and appropriate language model switching for all three tested languages. Figure 35 displays the language selection interface actively used on the multilingual test document.

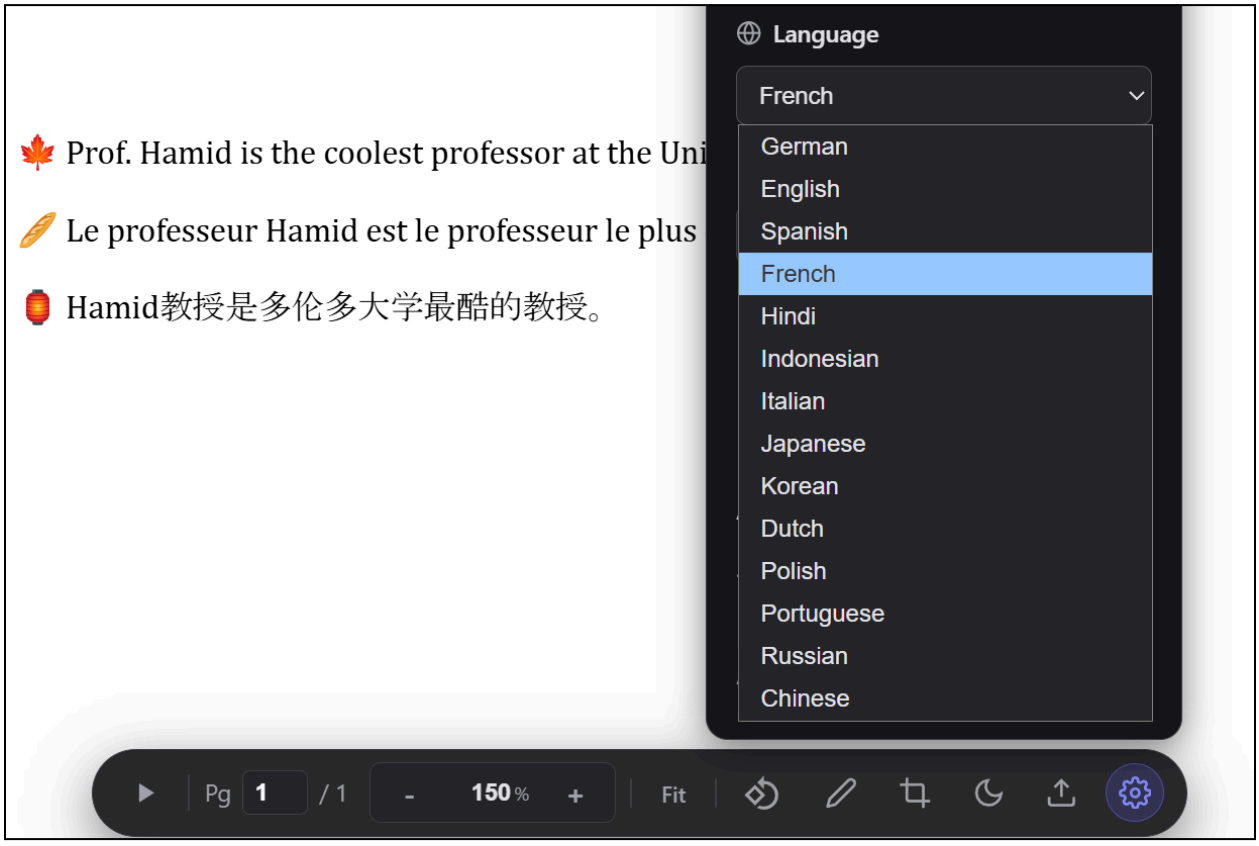


Figure 35. Application interface displaying a multilingual PDF document and the active language selection dropdown menu.

Appendix T13: Interaction Latency Verification

Test Objective: Verify the application maintains an Interaction to Next Paint (INP) latency of less than 96 ms to ensure interface responsiveness.

Test: UI responsiveness was measured using the Chrome DevTools Performance panel. Many user interactions, including pointer clicks and keyboard inputs, were executed and recorded.

Results: The system passed the latency threshold. Figure 36 displays the local metrics dashboard, confirming an overall INP value of 80 ms. Figure 37 shows the detailed interaction log, identifying the specific pointer event responsible for the 80 ms peak and demonstrating that all logged UI events executed below the 96 ms limit.

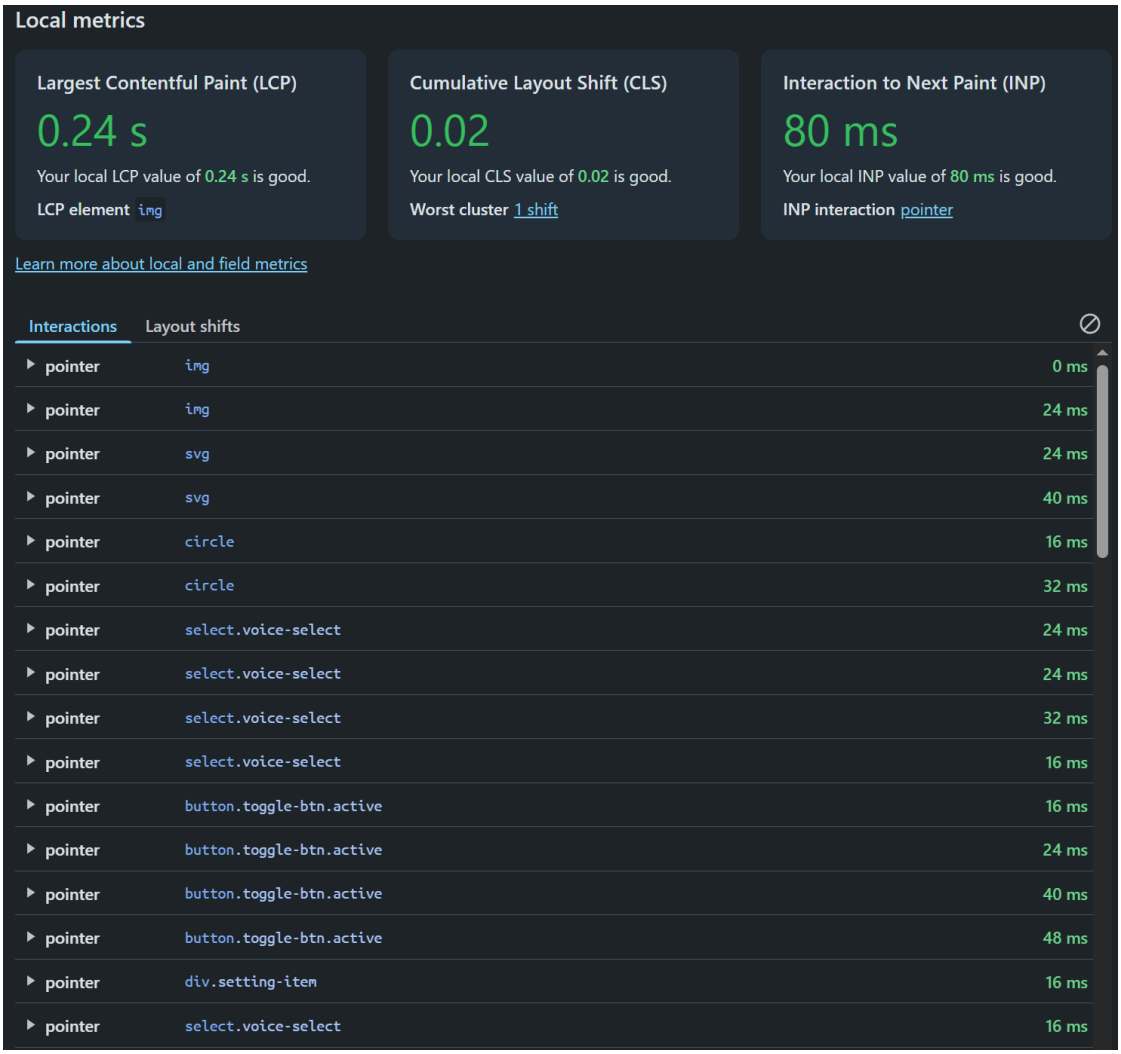


Figure 36. Chrome DevTools Performance panel showing an INP score of 80 ms alongside LCP and CLS metrics.

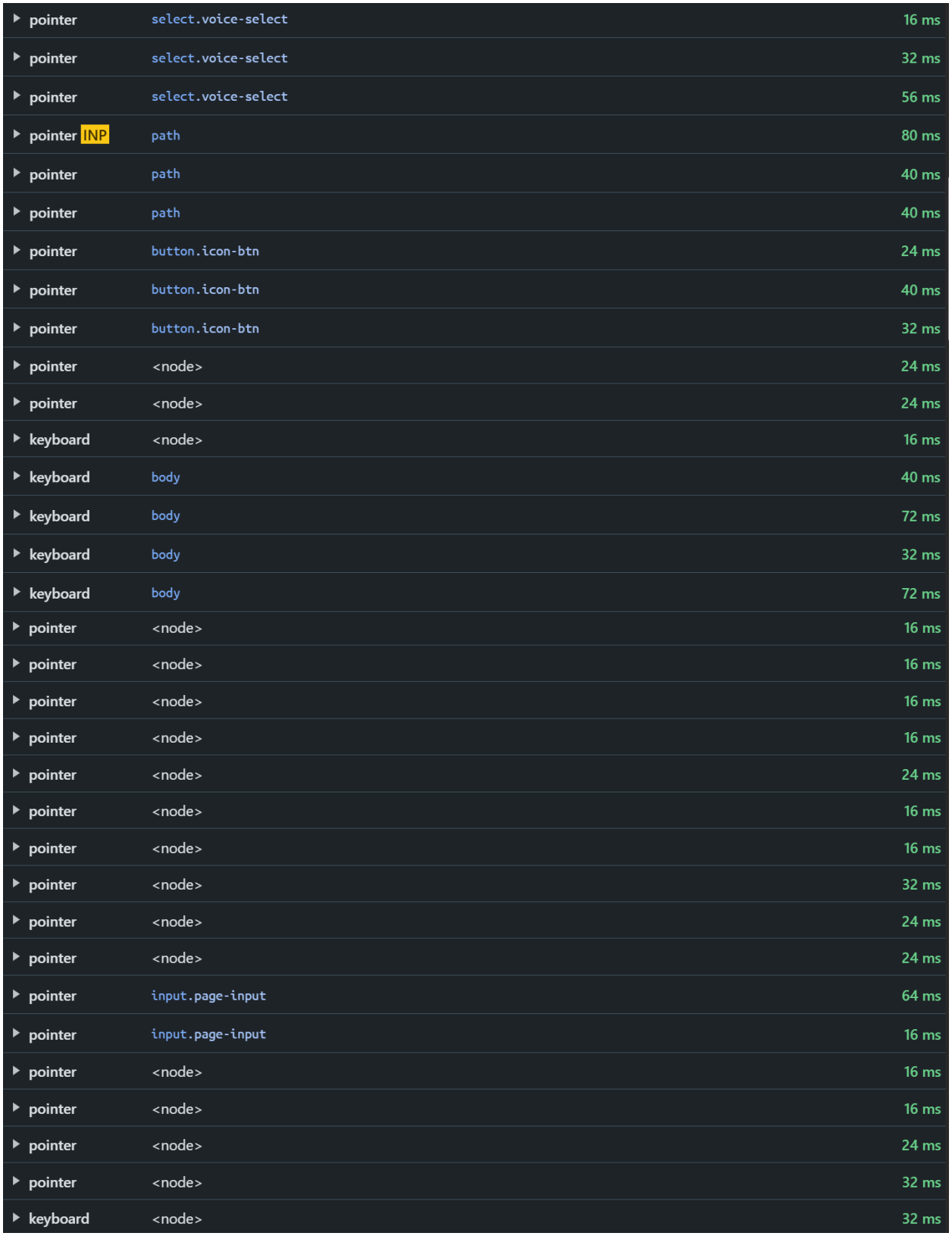


Figure 37. Chrome DevTools Performance panel, continued.

Appendix T14: Large File Reliability Verification

Test Objective: Verify the application can reliably ingest, parse, and process PDF documents up to and exceeding the 1 GB size limit .

Test: A 1.84 GB, 1446-page academic textbook DF was uploaded to the application. Total time from file selection to complete rendering was measured.

Results: The system exceeded the 1 GB requirement, successfully parsing the 1.84 GB document in 9.67 seconds. Figure 38 displays the file properties confirming the size, alongside the application interface showing the successfully loaded document and the recorded stopwatch time.

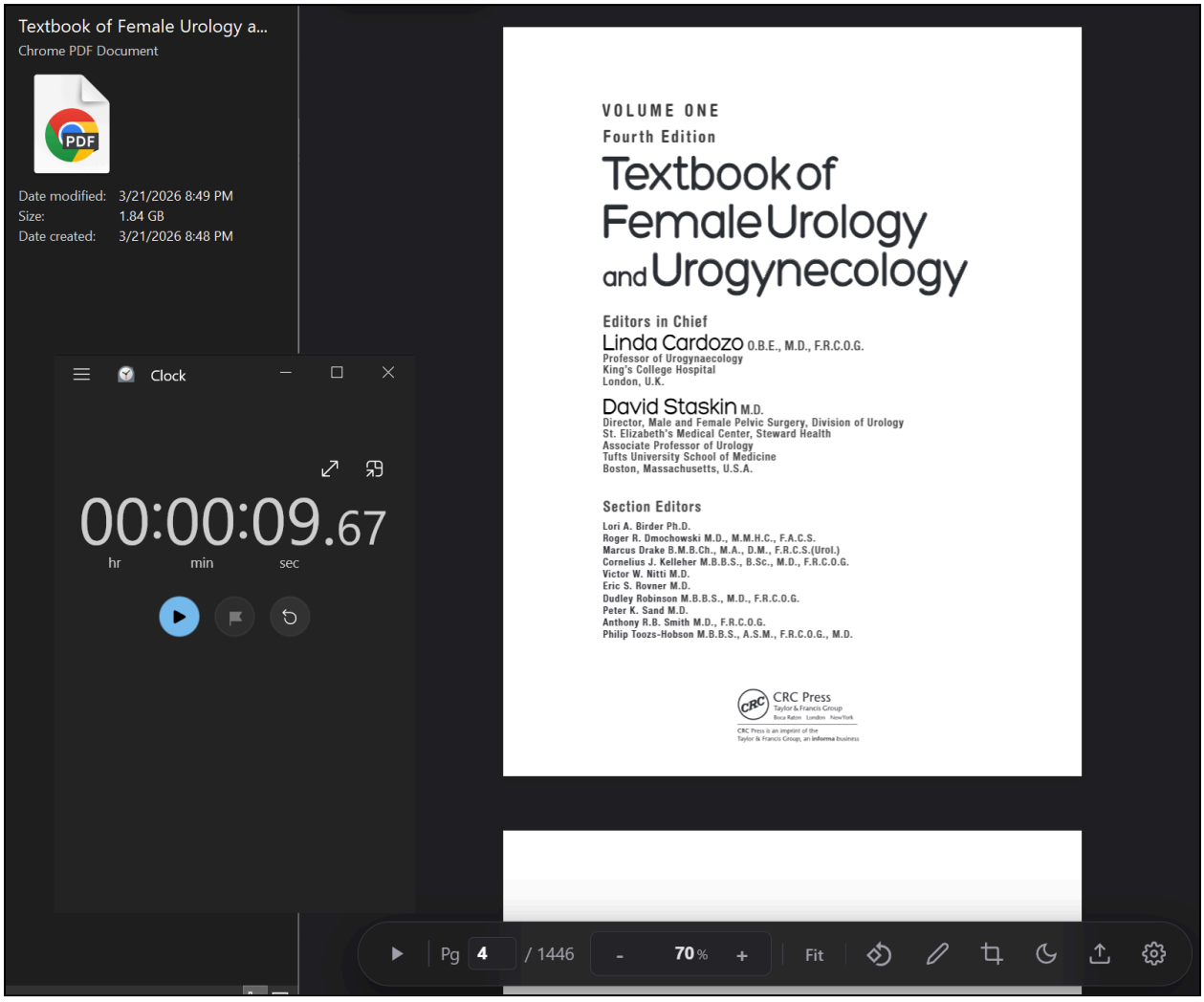


Figure 38. System file properties showing a 1.84 GB PDF and the application interface displaying the fully loaded document with a load time of 9.67 seconds.

Appendix T15: Open-Source License Compliance Verification

Test Objective: Verify all third-party software libraries used in the application comply with permissive open-source licensing requirements.

Test: Conducted an audit of the application's dependency tree, cross-referencing each core library against its published software license.

Results: The analysis confirmed all primary dependencies utilize permissive, OSI-approved licenses (MIT and Apache-2.0). Table V details the audited libraries, confirming no restrictive licenses are present in the core technology stack.

Table V: List of all software libraries, their corresponding open-source licenses and types.

Library Name	**License**	**Type**
Vite	MIT	Permissive
React	MIT	Permissive
Generative-ai (Google AI SDK)	Apache-2.0	Permissive
PDF.js	Apache-2.0	Permissive